



ĐẠI HỌC
BÁCH KHOA HÀ NỘI
HANOI UNIVERSITY
OF SCIENCE AND TECHNOLOGY

Code Generation with LLMs

Lecture: Le Hai Nam

namlh@soict.hust.edu.vn

ONE LOVE. ONE FUTURE.

Outline

1. Brief history of AI for code generation
2. Language Models in Code Generation
3. Pretraining CodeLLMs
4. Evaluation in Code Generation

1. A Brief History

Brief history of AI for code generation

In 2012...

Natural languages like English are rich, complex, and powerful. We begin with the conjecture that most software is also natural, in the sense that it is created by humans at work, with all the attendant constraints and limitations—and thus, like natural language, it is also likely to be repetitive and predictable. We then proceed to ask whether

- a. code can be usefully modeled by statistical language models**
- b. such models can be leveraged to support software engineers.**

"On the Naturalness of Software"

[Hindle et al., ICSE 2012. Most Influential Paper 2022]

<https://cs.ucdavis.edu/news/prem-devanbu-collaborators-receive-2022-icse-most-influential-paper-award>



In 2012...

- N-gram models trained on 14 million tokens of code
- Substantial improvements to Eclipse IDE's auto-complete
- Key differences:
 - 2-4 orders of magnitude less data than modern LLMs
 - Count-based n-gram vs. transformer

"On the Naturalness of Software"

[Hindle et al., ICSE 2012. Most Influential Paper 2022]

Brief history of AI for code generation

1. Source Code (C#):

```
public int TextWidth(string text) {  
    TextBlock t = new TextBlock();  
    t.Text = text;  
    return  
        (int)Math.Ceiling(t.ActualWidth);  
}
```

Descriptions:

- Get rendered width of string rounded up to the nearest integer
- Compute the actual textwidth inside a textblock

2. Source Code (C#):

```
var input = "Hello";  
var regex = new Regex("World");  
return !regex.IsMatch(input);
```

Descriptions:

- Return if the input doesn't contain a particular word in it
- Lookup a substring in a string using regex

```
Bitmap bitmap = BitmapFactory.decodeResource(getResources(),  
    R.drawable.image);
```

Example B: Post-Processing.

- This code snippet decodes a bitmap.³
- The recommended code suggests applying Gaussian blur on the decoded image, a customary effect to be applied.
- Intersected from a cluster of 4 methods.⁴

```
int radius = seekBar.getProgress();  
if (radius < 1) {  
    radius = 1;  
}  
Bitmap bitmap = BitmapFactory.decodeResource(getResources(),  
    R.drawable.image);  
imageView.setImageBitmap(blur.gaussianBlur(radius, bitmap));
```

Code summarization in Java:

```
public boolean ① (Set<String> ② set,  
    String value) {  
    for (String ③ entry : set) {  
        if (entry.equalsIgnoreCase(value))  
            return true;  
    }  
    return false;  
}
```

contains^① ignore^② case^③

CodeNN (Iyer et al., 2016)
Code Summarization

Aroma (Luan et al, 2019)
Early Copilot

Code2Seq (Alon et al., 2019)
Better code summarization

Pre-LLM era: RNNs
and search

Early applications:
GPT-3, Codex, GitHub Copilot

"Oh wow, AI can actually write code
now": GPT-3.5, GPT-4, OSS LLMs

AI x software engineering:
Agents and integrated workflows



Brief history of AI for code generation



GitHub Copilot

Codex (Chen, Mark, et al, 2021)



Pre-LLM era: RNNs and search

Early applications:
GPT-3, Codex, GitHub Copilot

“Oh wow, AI can actually write code now”: GPT-3.5, GPT-4, OSS LLMs

AI x software engineering:
Agents and integrated workflows

Brief history of AI for code generation

ChatGPT

Write me a program in python to train an LM using transformers library

Here's a basic Python script to train a language model using the `transformers` library from Hugging Face. This example fine-tunes a pretrained model (such as `bert-base-uncased`) on a custom text dataset. The script uses `Trainer` for ease of training.

Train Lm Transformers

Answer in chat instead >

Explanation:

- Load a Pretrained Model:** Uses `bert-base-uncased` as the base model for masked language modeling (MLM).
- Prepare Dataset:** Loads `wikitext-2-raw-v1` from Hugging Face Datasets,

Train Lm Transformers

```
1 import torch
2 from transformers import AutoModelForMaskedLM, AutoTokenizer, Trainer, TrainingArguments, DataCollatorForLanguageModeling
3 from datasets import load_dataset
4
5 # Define model and tokenizer
6 MODEL_NAME = "bert-base-uncased"
7 model = AutoModelForMaskedLM.from_pretrained(MODEL_NAME)
8 tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
9
10 # Load dataset
11 dataset = load_dataset("wikitext", "wikitext-2-raw-v1")
12
13 def tokenize_function(examples):
14     return tokenizer(examples["text"], truncation=True, padding="max_length", max_length=128)
15
16 tokenized_datasets = dataset.map(tokenize_function, batched=True, remove_columns=["text"])
17
18 data_collator = DataCollatorForLanguageModeling(tokenizer=tokenizer, mlm=True, mlm_probability=0.15)
19
20 # Define training arguments
21 training_args = TrainingArguments(
22     output_dir="./lm_model",
23     evaluation_strategy="epoch",
24     save_strategy="epoch",
25     per_device_train_batch_size=8,
26     per_device_eval_batch_size=8,
27     num_train_epochs=3,
28     weight_decay=0.01,
29     save_total_limit=2,
30     logging_dir="./logs".
```

Pre-LLM era: RNNs and search

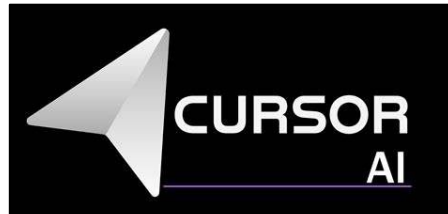
Early applications: GPT-3, Codex, GitHub Copilot

"Oh wow, AI can actually write code now": GPT-3.5, GPT-4, OSS LLMs

AI x software engineering: Agents and integrated workflows



Brief history of AI for code generation



● GPT-Migrate ●



OpenHands: Code Less, Make More

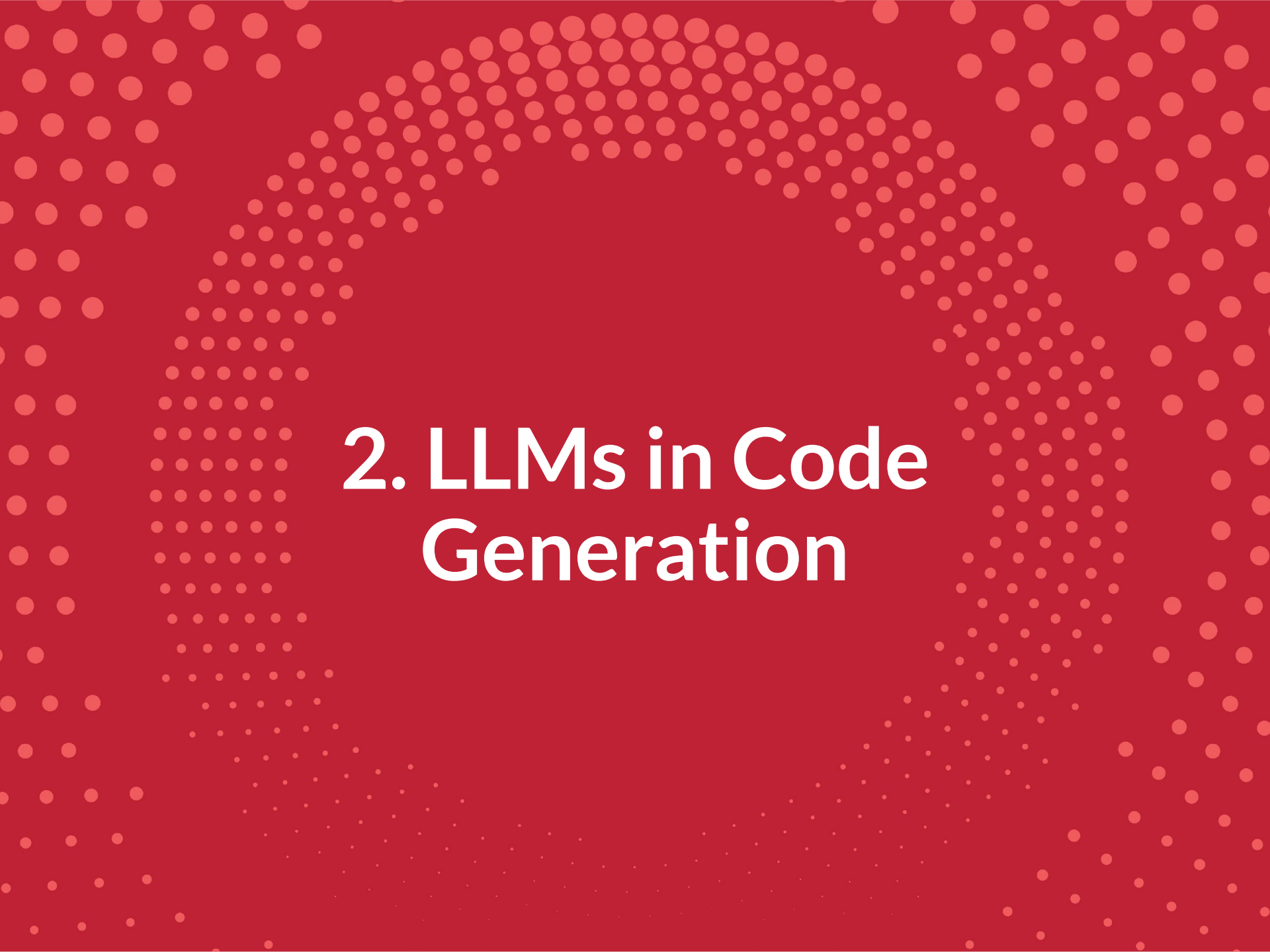


Pre-LLM era: RNNs
and search

Early applications:
GPT-3, Codex, GitHub Copilot

“Oh wow, AI can actually write code
now”: GPT-3.5, GPT-4, OSS LLMs

AI x software engineering:
Agents and integrated workflows

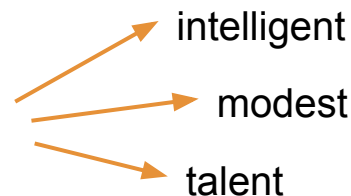


2. LLMs in Code Generation

Language Modeling

- **Language Modeling** is the task of predicting what word comes at a specific position.

Students at HUST are very _____



- More formally: given a sequence of words $x^{(1)}, x^{(2)}, \dots, x^{(t)}$ compute the probability distribution of the next word $x^{(t+1)}$:

$$P\left(x^{(t+1)} \mid x^{(t)}, \dots, x^{(1)}\right)$$

where $x^{(t+1)}$ can be any word in the vocabulary $V = \{w_1, \dots, w_{|V|}\}$

- A system that does this is called a **Language Model**.

NL to Code generation

Problem

H-Index

Given a list of citations counts, where each citation is a nonnegative integer, write a function `h_index` that outputs the h-index. The h-index is the largest number h such that h papers have each least h citations.

Example:

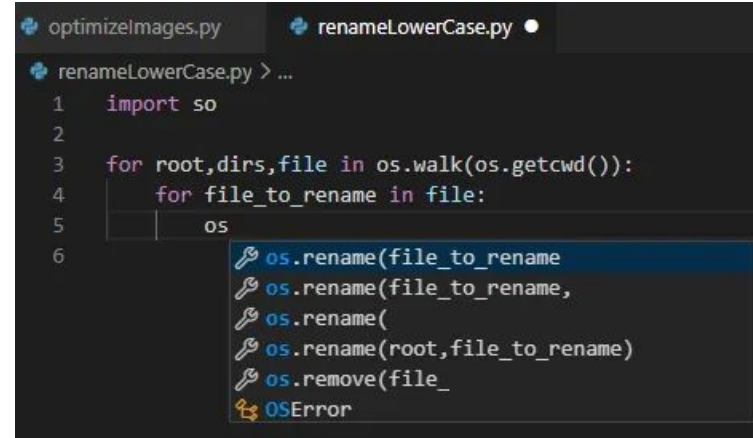
Input: [3,0,6,1,4]

Output: 3

Generated Code

```
def h_index(counts):
    n = len(counts)
    if n > 0:
        counts.sort()
        counts.reverse()
        h = 0
        while (h < n and
               counts[h]-1 >= h):
            h += 1
        return h
    else:
        return 0
```

Code Completion



The screenshot shows a code editor with two files: `optimizeImages.py` and `renameLowerCase.py`. The `renameLowerCase.py` file is open, showing the following code:

```
1 import os
2
3 for root,dirs,file in os.walk(os.getcwd()):
4     for file_to_rename in file:
5         os
6
```

A dropdown menu is visible under the `os` variable on line 5, showing suggestions for `os.rename(file_to_rename`, `os.rename(file_to_rename,`, `os.rename(`, `os.rename(root,file_to_rename)`, `os.remove(file_`, and `OSError`.

Language Models for Code

- Early language models for code
 - N-gram language models [Hindle et al 2012, Allamanis & Sutton 2013]

$$p(a_4|a_1a_2a_3) = \frac{\text{count}(a_1a_2a_3a_4)}{\text{count}(a_1a_2a_3*)}$$

- Latent predictor network [Ling et al 2016]: seq2seq architecture for code generation



```
class MadderBomber(MinionCard):    BLEU = 100.0
    def __init__(self):
        super().__init__("Madder Bomber", 5,
            CHARACTER_CLASS.ALL, CARD_RARITY.RARE,
            battlecry=Battlecry(Damage(1),
                CharacterSelector(players=BothPlayer(),
                    picker= RandomPicker(6))))
    def create_minion(self, player):§
        return Minion(5, 4)§
```

Fig. Generate code from a description of a card

Language Models for Code

- Transformer-based LMs

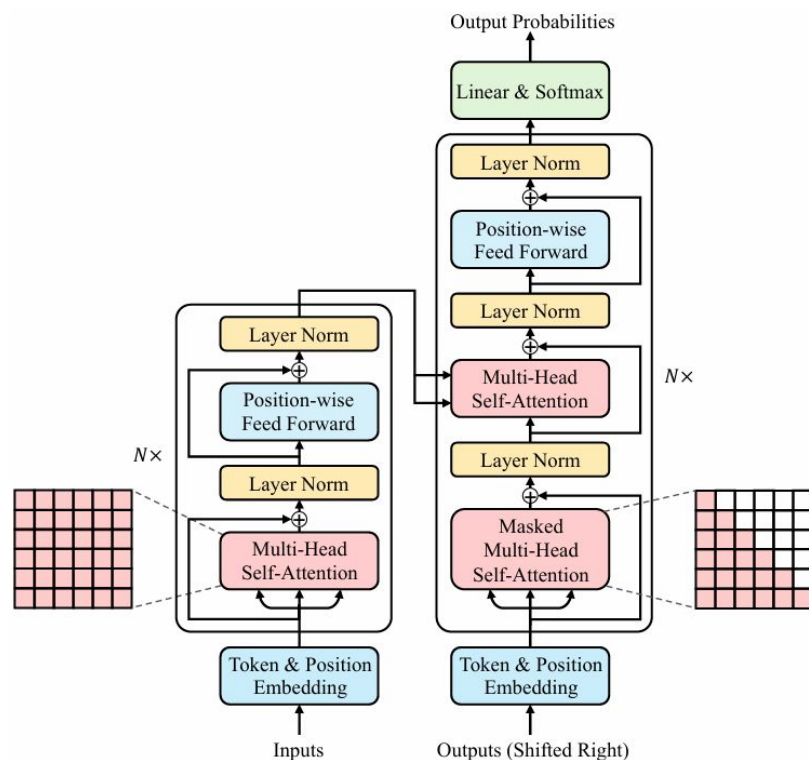
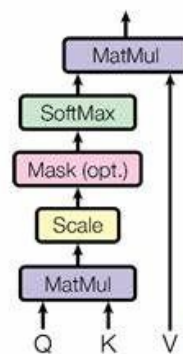


Fig. Transformer architecture
(Jiang, Juyong, et al., 2024)

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_{\text{model}}/h}}\right)V,$$

Scaled Dot-Product Attention



Multi-Head Attention

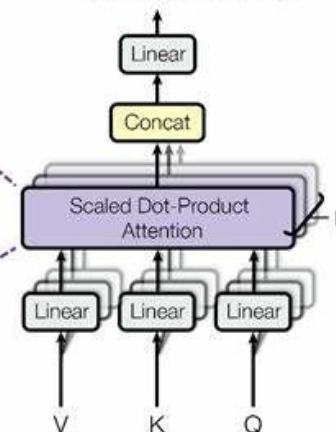


Fig. Self-attention
(Vaswani, Ashish, et al, 2017)

Language Models for Code

- Transformer-based LMs

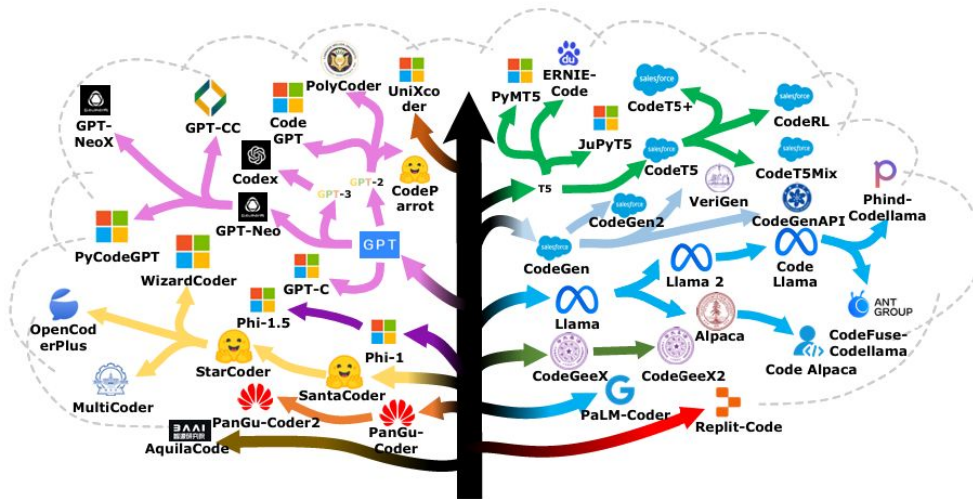
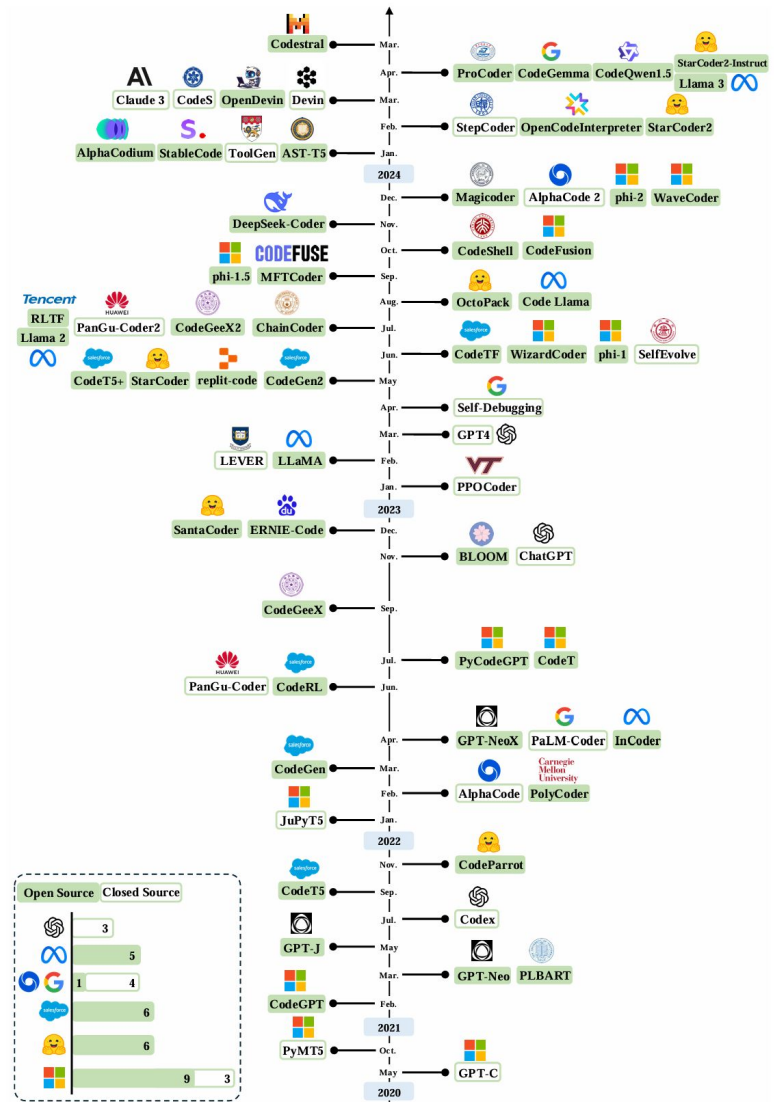


Fig. The relationships between Code LLMs
(Zheng, Zibin, et al. 2023)

- Large language models (LLMs) are language models with many parameters (billions of parameters)
- **CodeLLMs:** LLMs that are trained on code data

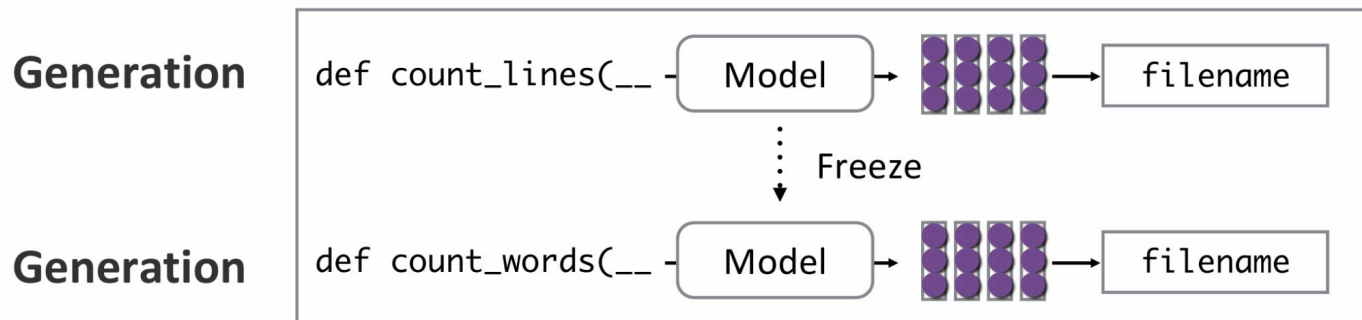


Jiang, et al. "A survey on large language models for code generation." (2024)

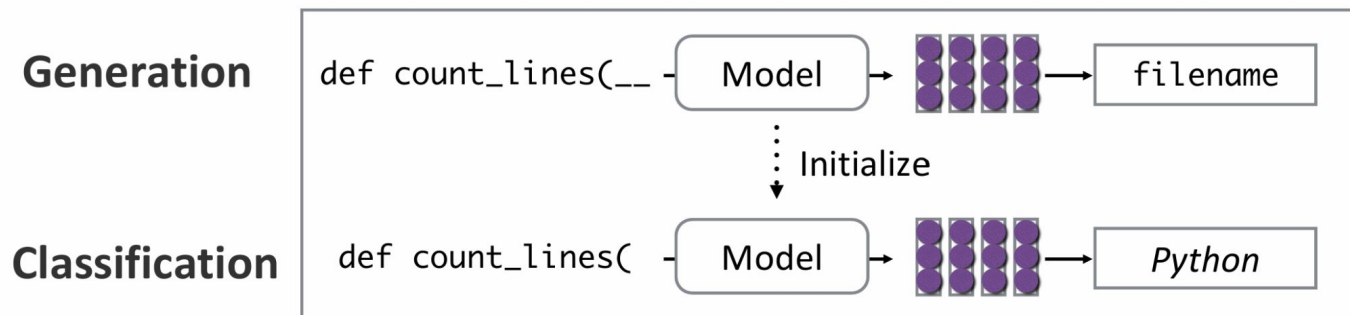
3. Pretraining CodeLLMs

Scenarios

- Train the model to generate language/code, then use --without updating the model on other generation tasks.



- First train on one task, then train on another



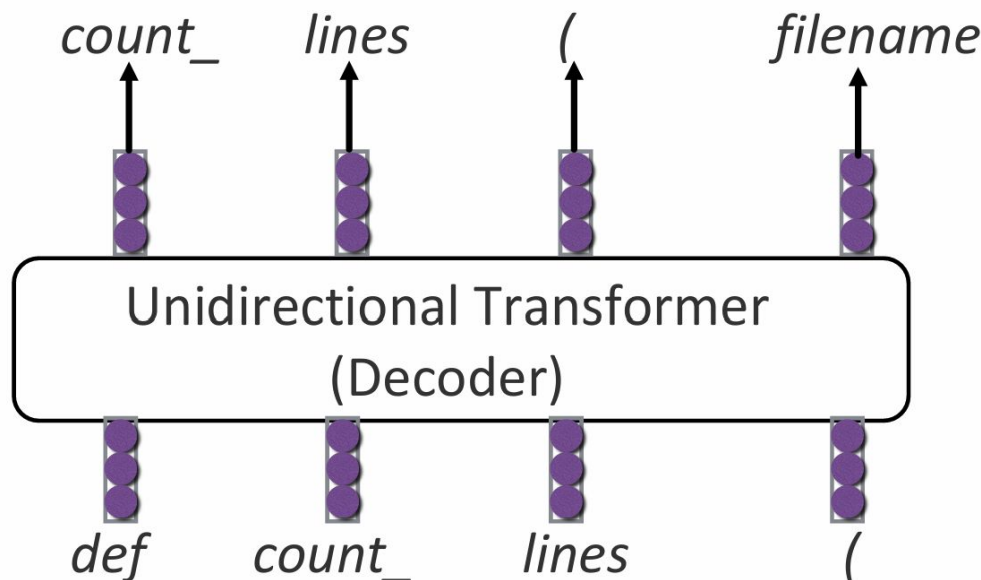
Objectives: Autoregressive Language Modeling

$$P(X) = \prod_{i=1}^{|X|} P(x_i | x_1, \dots, x_{i-1})$$

Outputs:

Used mostly for
generation/prompting

Inputs:



Autoregressive Language Modeling

Masked Language Modeling

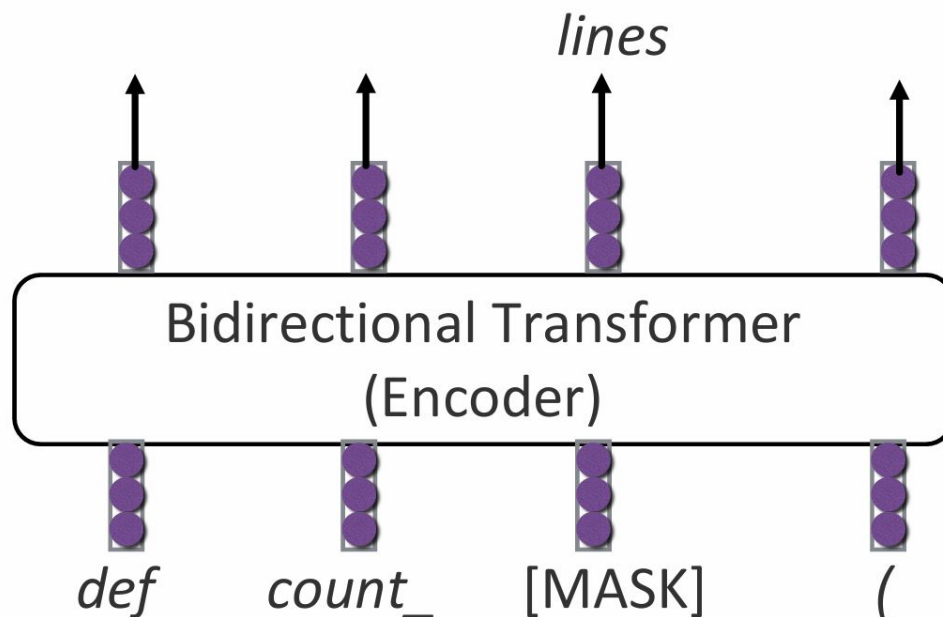
Sequence-to-sequence

Objectives: Masked Language Modeling

$$P(X) \neq \prod_{i=1}^{|X|} P(x_i | x_{\neq i})$$

Outputs:

Used mostly for
representation learning



Inputs:

Autoregressive Language Modeling

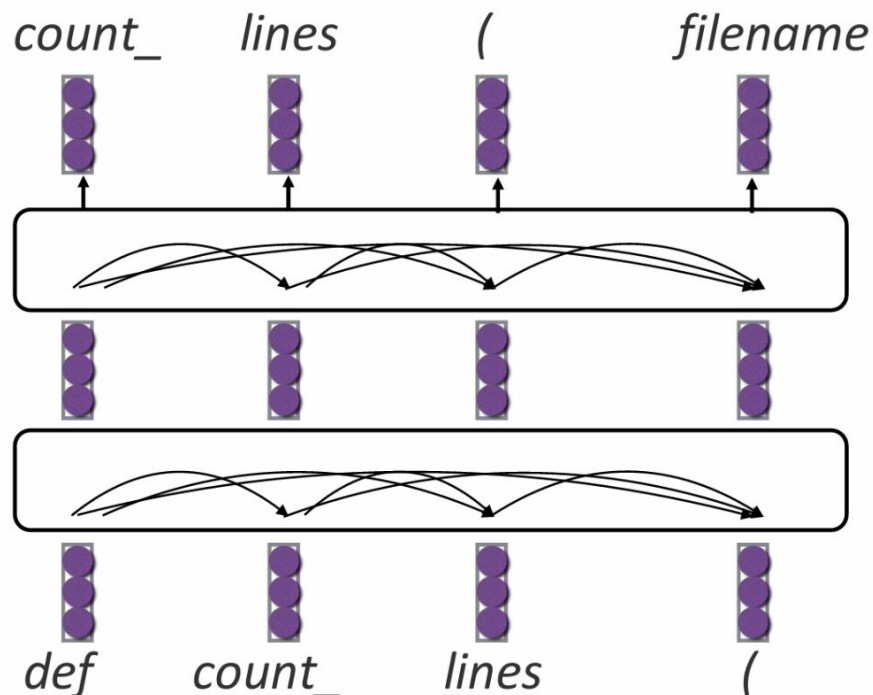
Masked Language Modeling

Sequence-to-sequence

Unidirectional vs Bidirectional Transformers

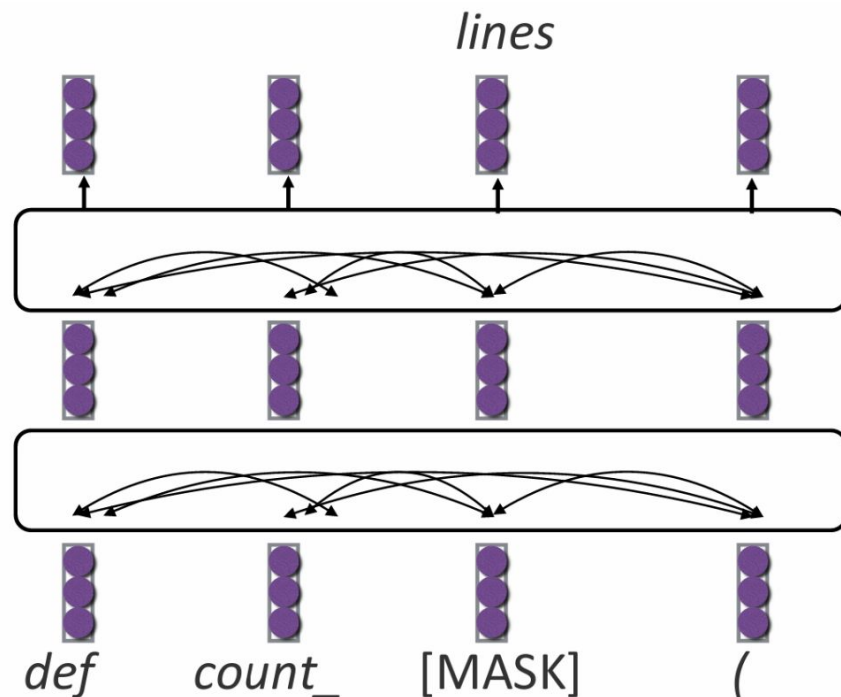
Unidirectional

Each token has info about previous.



Bidirectional

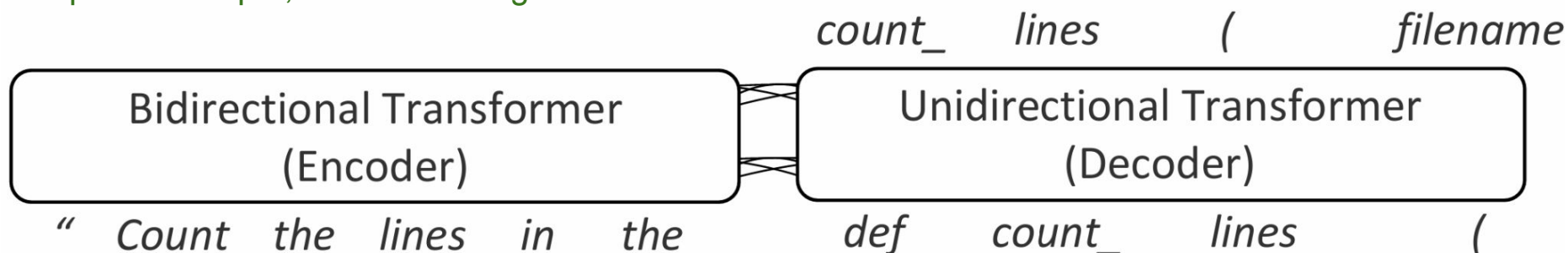
Each token has info about all others.



Objectives: Sequence-to-Sequence

$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i|X, y_1, \dots, y_{i-1})$$

Used mostly for tasks with explicit input and output, with fine-tuning



Autoregressive Language Modeling

Masked Language Modeling

Sequence-to-sequence

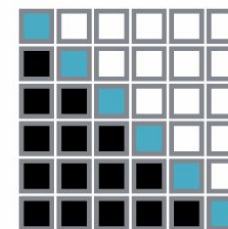
Which Objective?

Decoder-only

Autoregressive language modeling

$$P(X) = \prod_{i=1}^{|X|} P(x_i | x_1, \dots, x_{i-1})$$

used more for prompting/text generation

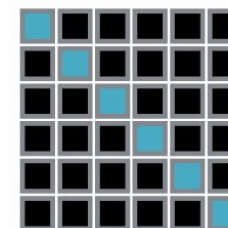


Encoder-only

Masked language modeling

$$P(X) \neq \prod_{i=1}^{|X|} P(x_i | x_{\neq i})$$

used more for pre-training + fine-tuning

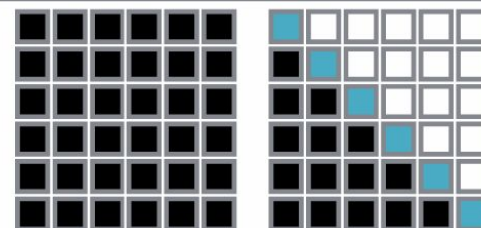


Encoder-Decoder

Seq-to-seq de-noising

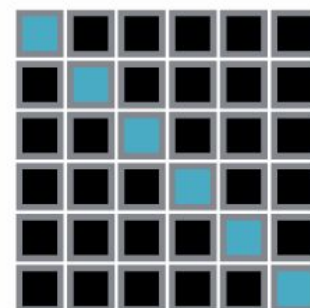
$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i | X, y_1, \dots, y_{i-1})$$

used for pre-train + fine-tune on generation tasks



Masked Language Modeling

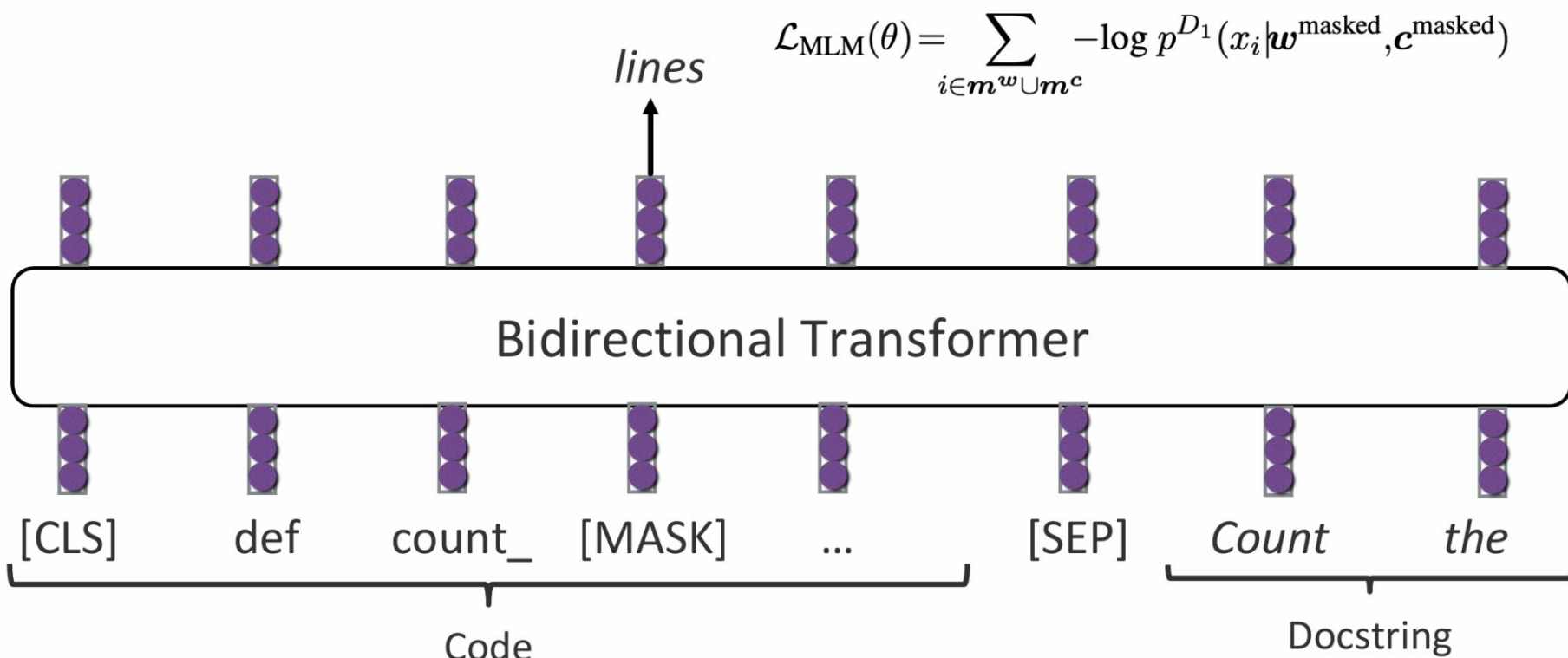
$$P(X) \neq \prod_{i=1}^{|X|} P(x_i | x_{\neq i})$$



used more for pre-training + fine-tuning

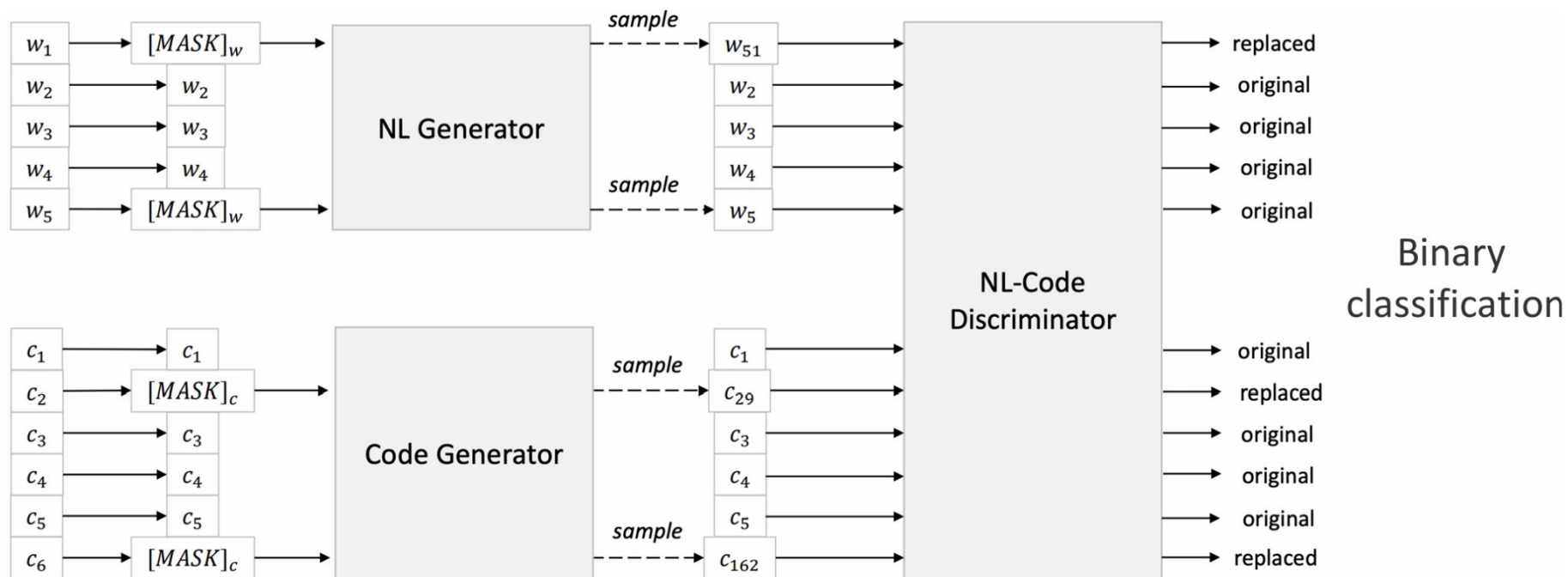
CodeBERT (Feng, et al. 2020)

Mask 15% of the tokens, randomly, and try to predict these masked tokens.



CodeBERT: Replaced Token Detection Objective

Rather than masked tokens, use tokens replaced by (weaker) LMs, and distinguish original tokens from replaced tokens.



CodeBERT: Pretraining

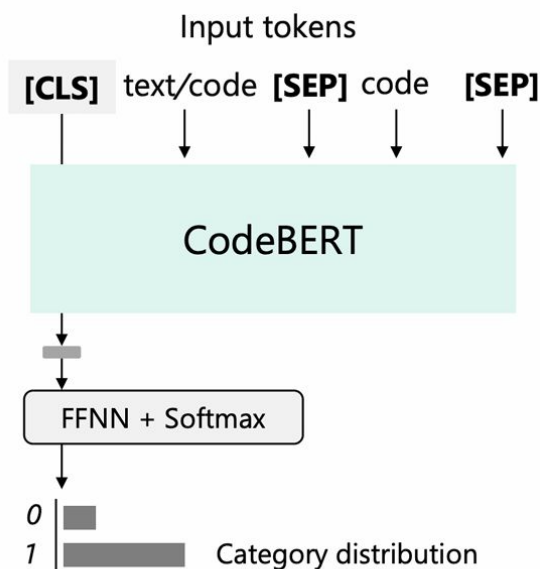
- ❖ 125M parameter bidirectional encoder Transformer
- ❖ Train on 2M documented functions (text & code) and 6M undocumented functions (code only) from GitHub (CodeSearchNet)

TRAINING DATA	<i>bimodal</i> DATA	<i>unimodal</i> CODES
Go	319,256	726,768
JAVA	500,754	1,569,889
JAVASCRIPT	143,252	1,857,835
PHP	662,907	977,821
PYTHON	458,219	1,156,085
RUBY	52,905	164,048
ALL	2,137,293	6,452,446

CodeBERT: Finetuning

- ❖ Parts of the task network are initialized with CodeBERT parameters.

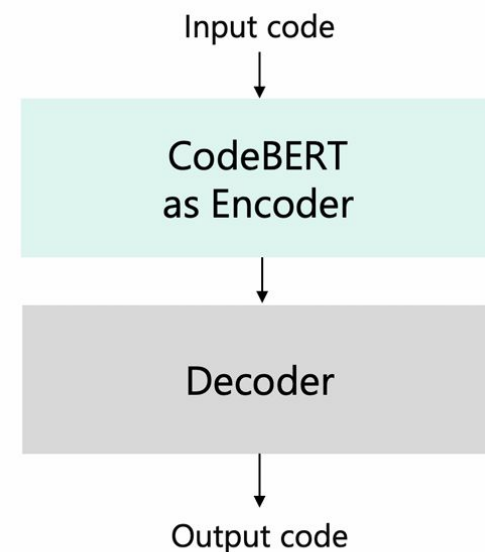
Classification Tasks



Supported tasks:

- code search
- code clone detection

Generation Tasks



Supported tasks:

- code repair
- code translation

CodeBERT: Results

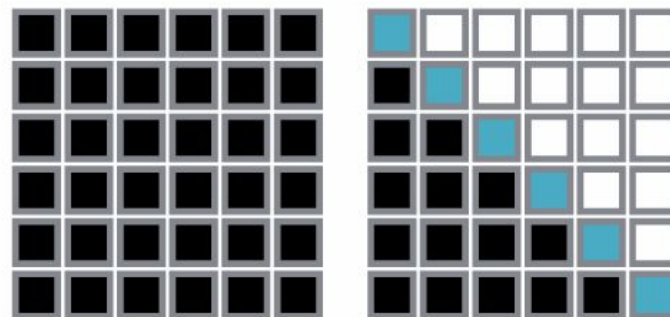
- ❖ Joint training on code and documentation > code alone
- ❖ Initializing with a text-only model (RoBERTa) helps

MODEL	RUBY	JAVASCRIPT	GO	PYTHON	JAVA	PHP	MA-AVG
RoBERTa	0.6245	0.6060	0.8204	0.8087	0.6659	0.6576	0.6972
PT W/ CODE ONLY (INIT=S)	0.5712	0.5557	0.7929	0.7855	0.6567	0.6172	0.6632
PT W/ CODE ONLY (INIT=R)	0.6612	0.6402	0.8191	0.8438	0.7213	0.6706	0.7260
CODEBERT (MLM, INIT=S)	0.5695	0.6029	0.8304	0.8261	0.7142	0.6556	0.6998
CODEBERT (MLM, INIT=R)	0.6898	0.6997	0.8383	0.8647	0.7476	0.6893	0.7549
CODEBERT (RTD, INIT=R)	0.6414	0.6512	0.8285	0.8263	0.7150	0.6774	0.7233
CODEBERT (MLM+RTD, INIT=R)	0.6926	0.7059	0.8400	0.8685	0.7484	0.7062	0.7603

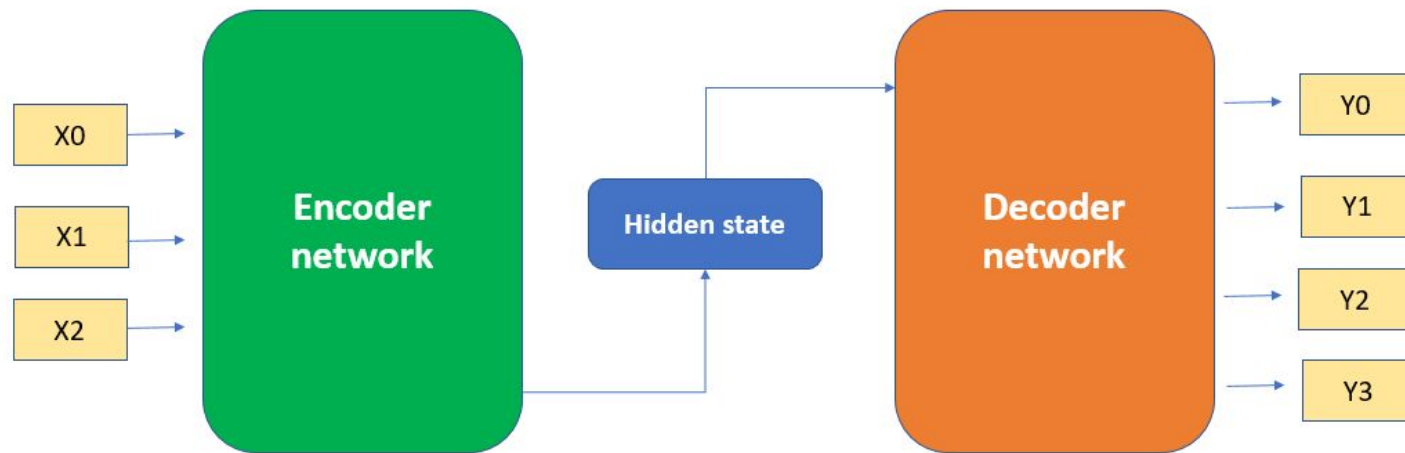
Results for function/documentation matching (code retrieval)

Encoder-Decoder LMs

$$P(Y|X) = \prod_{i=1}^{|Y|} P(y_i|X, y_1, \dots, y_{i-1})$$

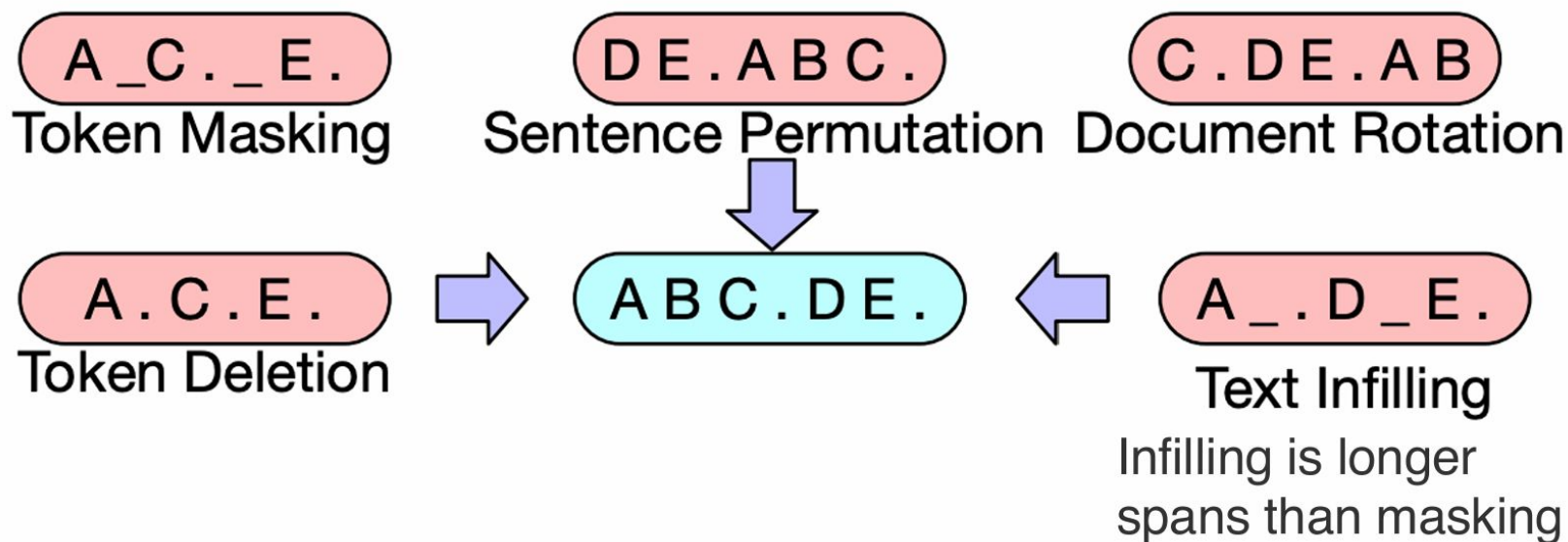


used for pre-train + fine-tune on generation tasks



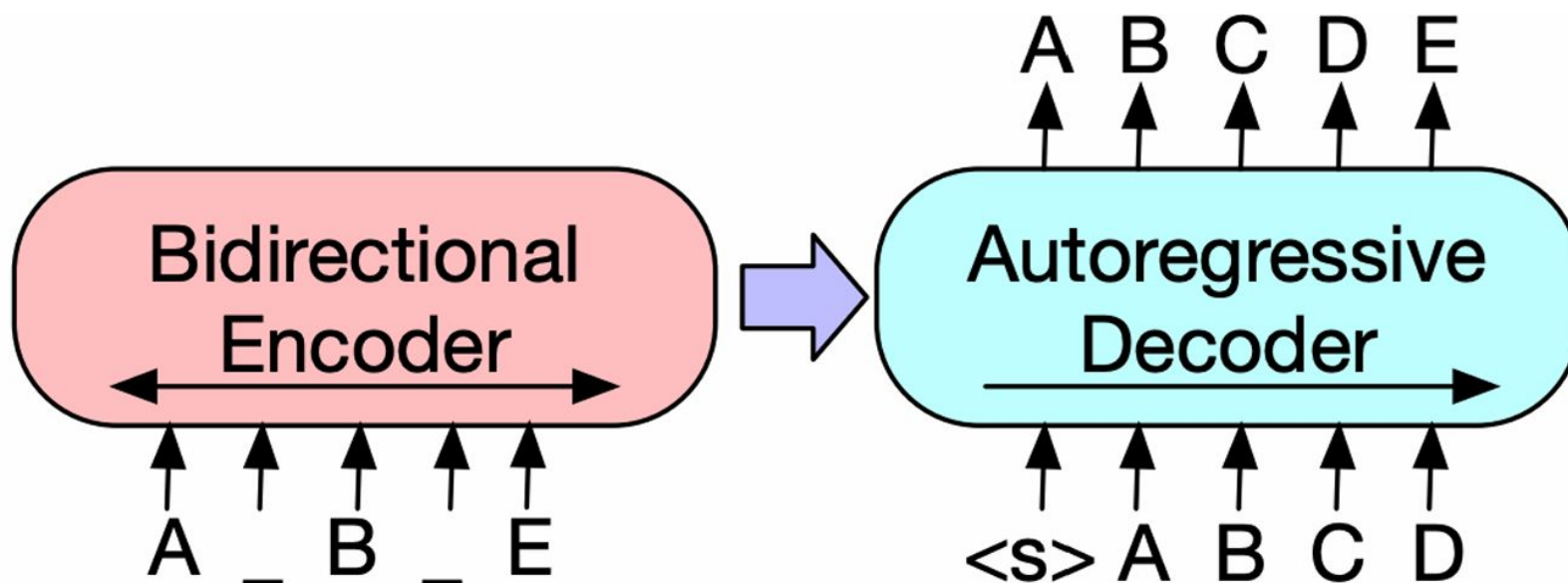
- Require input and output sequences => how can we pretrain from large raw/unlabeled text?

BART (Lewis, et al. 2019)



- Several possible strategies for corrupting a sequence are explored in the BART paper

BART (Lewis, et al. 2019)



- **Model & Objective:** Sequence-to-sequence Transformer trained on this data: permute/make/delete tokens, then predict full sequence autoregressively.
- **Data:** Same as RoBERTa; 160 GB of text

T5: Text-to-Text Transfer Transformer (Raffel et al. 2019)

- ❖ Objective: similar denoising scheme to BART (they were released within a week of each other in fall 2019).
- ❖ Input: text with gaps. Output: a series of phrases to fill those gaps.
- ❖ Lower computational cost compared to BART: predicts fewer tokens.
- ❖ Multi-task training

Original text

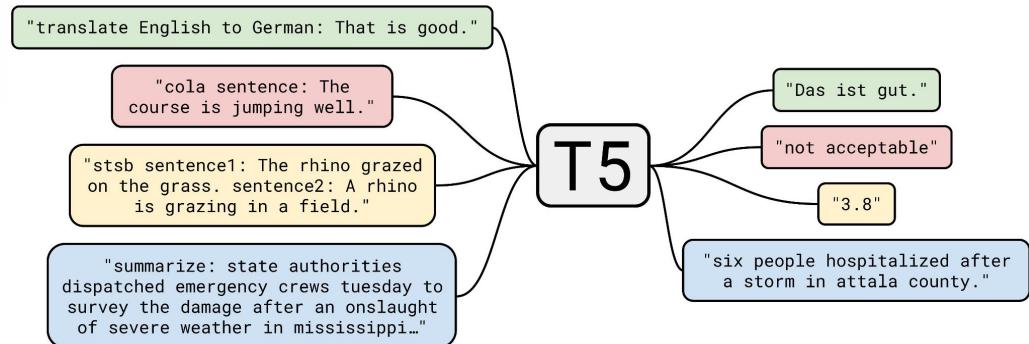
Thank you ~~for inviting~~ me to your party ~~last~~ week.

Inputs

Thank you <X> me to your party <Y> week.

Targets

<X> for inviting <Y> last <Z>



CodeT5 (Wang et al. 2021)

- ❖ Pre-train like T5 (seq-to-seq denoising/masked span prediction), but add identifier specific objectives to learn code semantics.

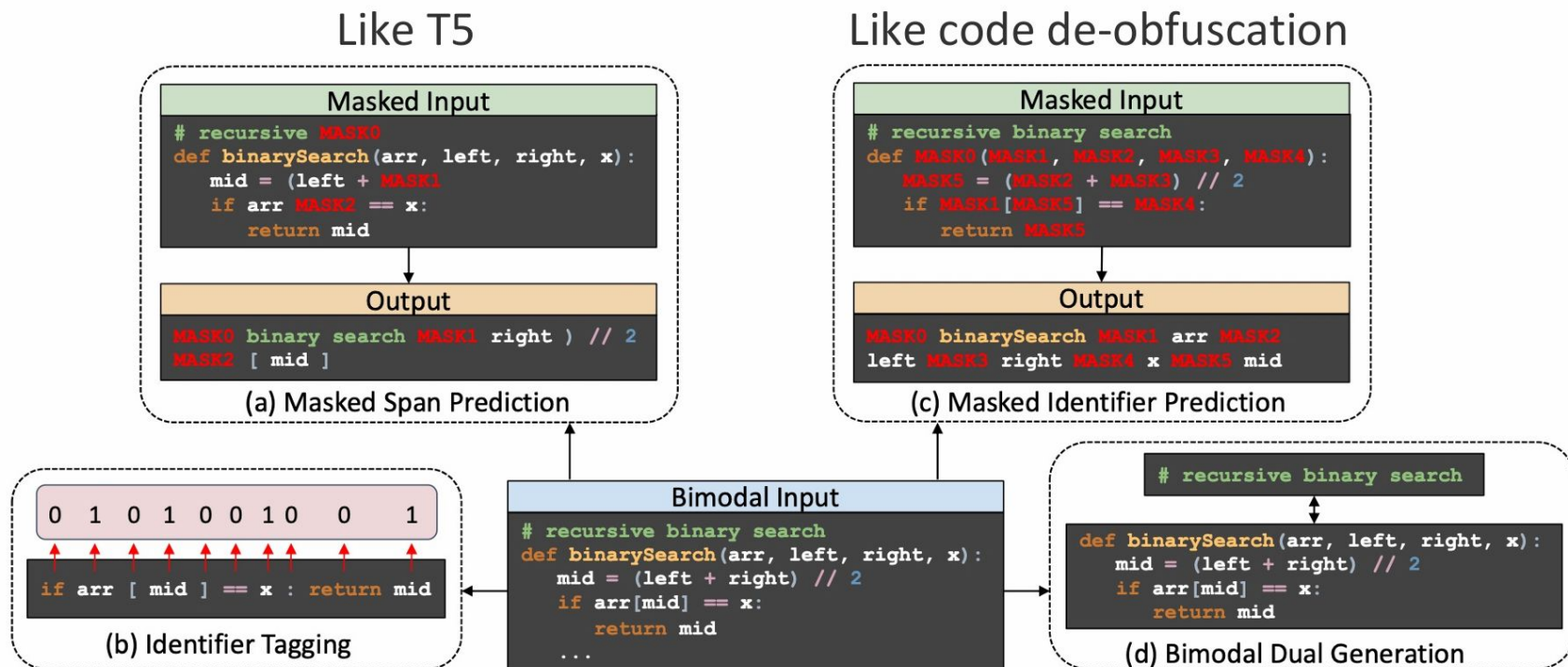
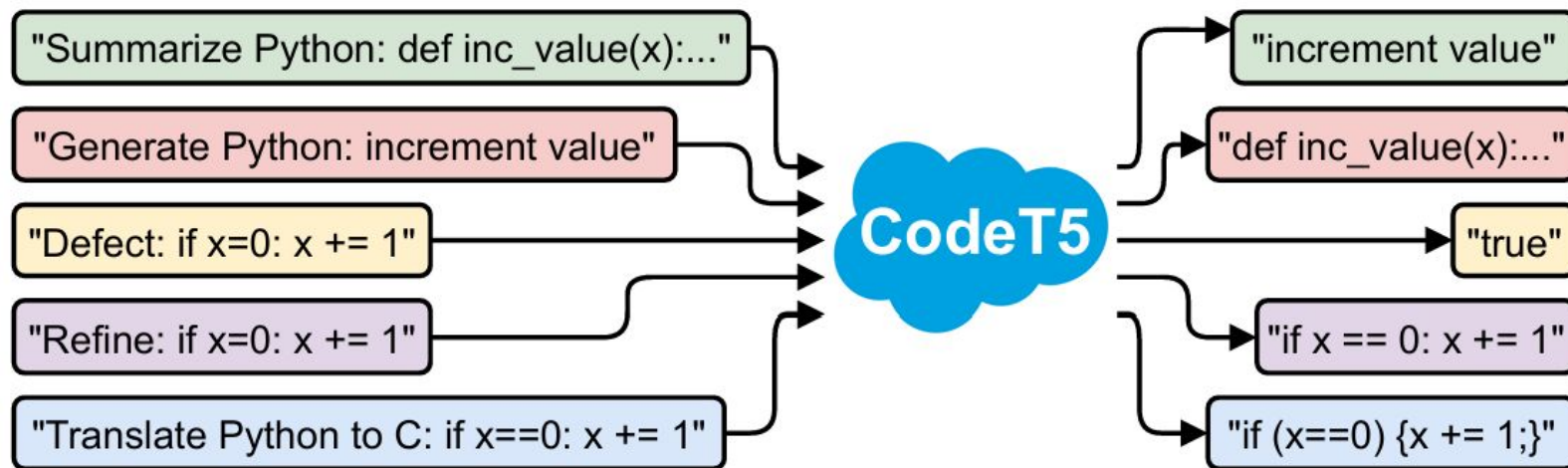


Figure 2: Pre-training tasks of CodeT5. We first alternately train span prediction, identifier prediction, and identifier tagging on both unimodal and bimodal data, and then leverage the bimodal data for dual generation training.

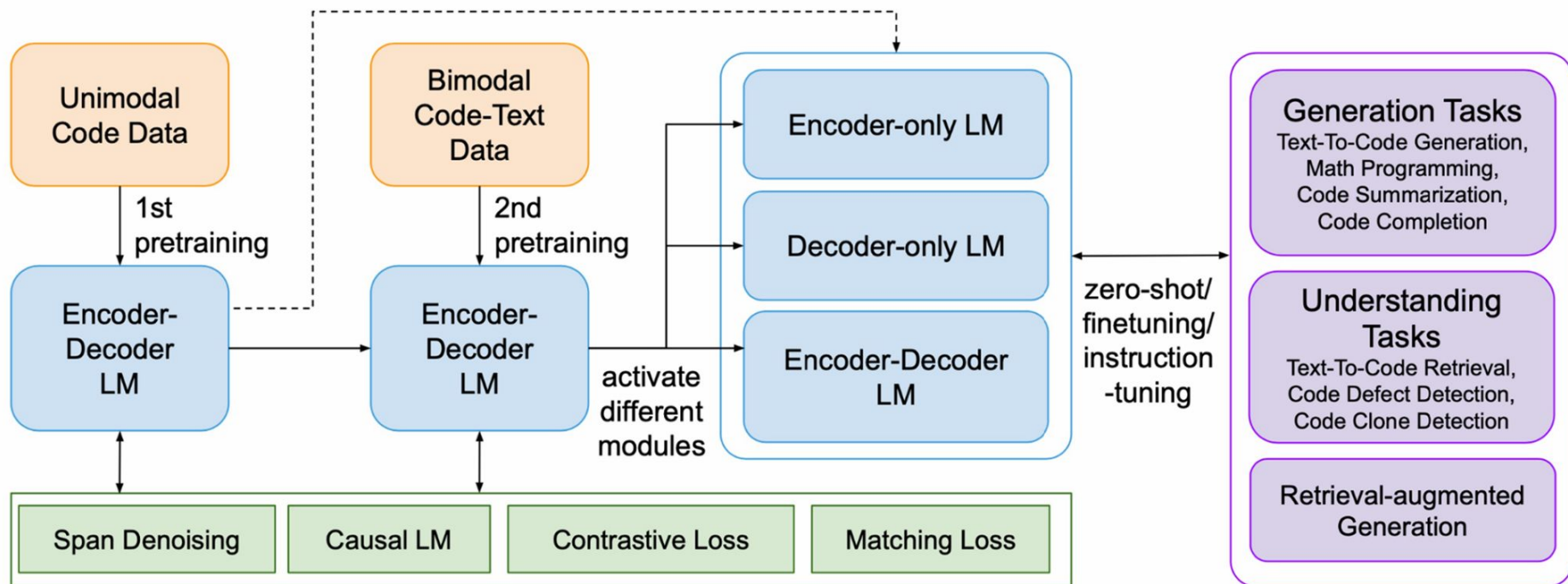
CodeT5 (Wang et al. 2021)

- Pre-train on CodeSearchNet (6 PLs) + BigQuery (C & C#); 8.4M instances
 - 60M and 220M parameter models, trained for 5 & 12 days on 16 GPUs.
 - Couldn't initialize with T5, because T5's tokenizer doesn't preserve code-specific symbols like { and }.
- Then, optionally do multi-task fine-tuning: train on multiple seq-to-seq tasks from CodeXGLUE simultaneously (translation, refinement, summarization)

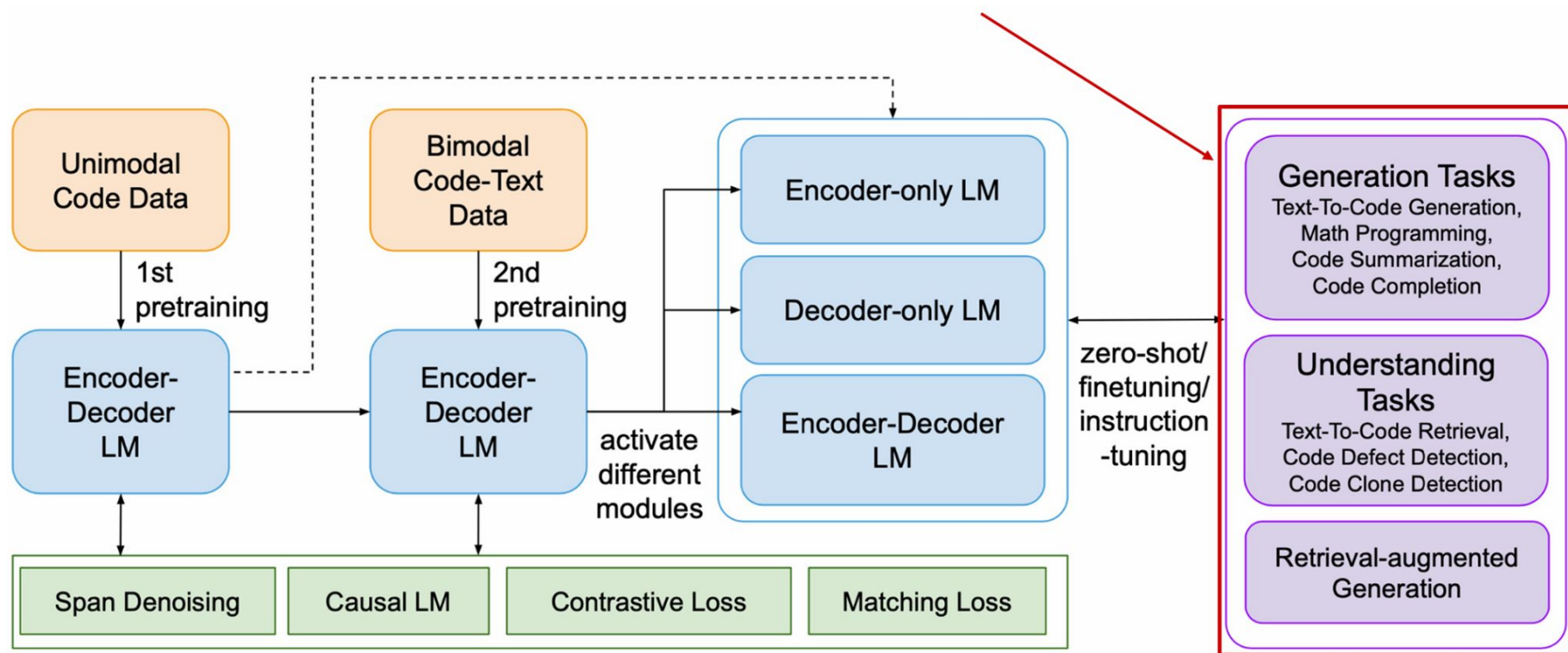


- Specializations of past approaches:
 - *For translation:* T5-like (seq-to-seq denoising) generally best
 - *For generating new content:* GPT-like (unidirectional decoder-only) generally best
 - *For doc-level embeddings:* BERT-like (MLM bidirectional encoder) generally best
- CodeT5+: use a seq-to-seq model but train it with a progression of objectives, and pre-trained initializations

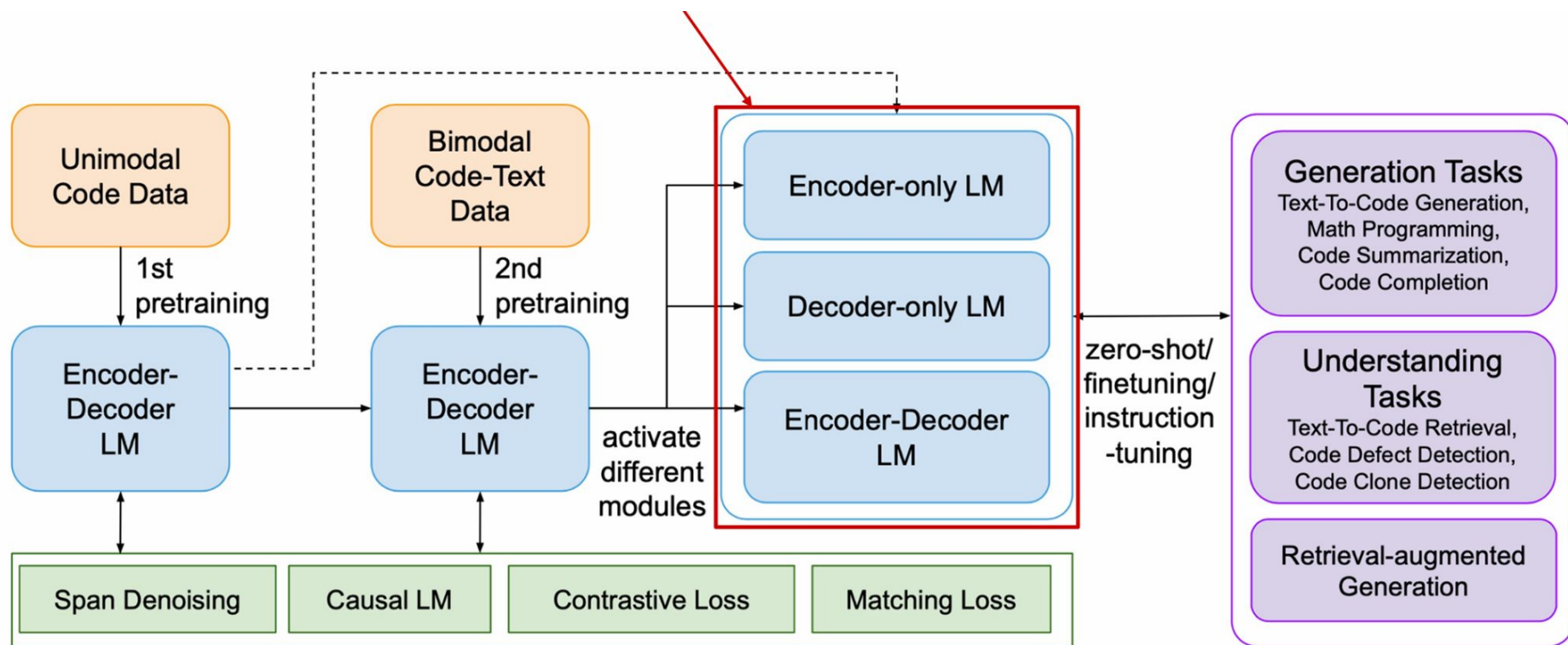
CodeT5+: Overview



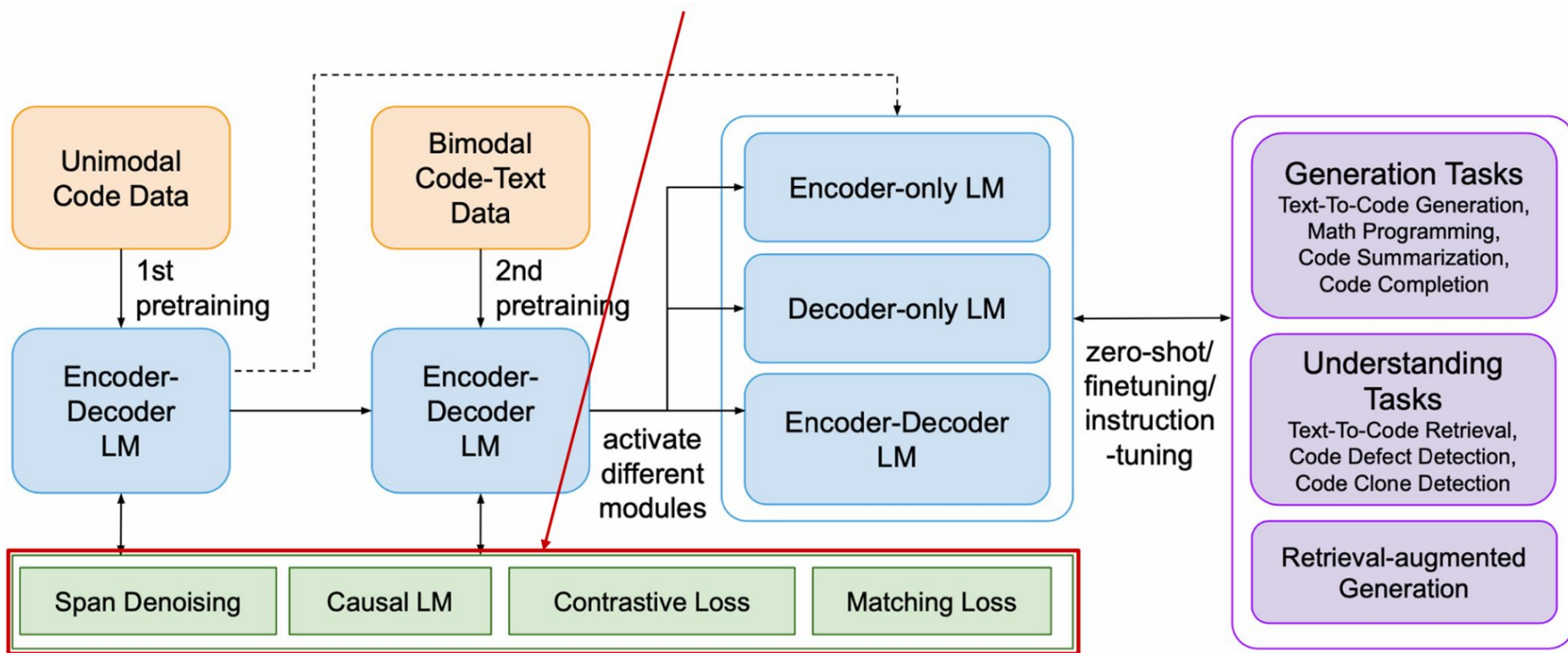
CodeT5+: Supports downstream tasks



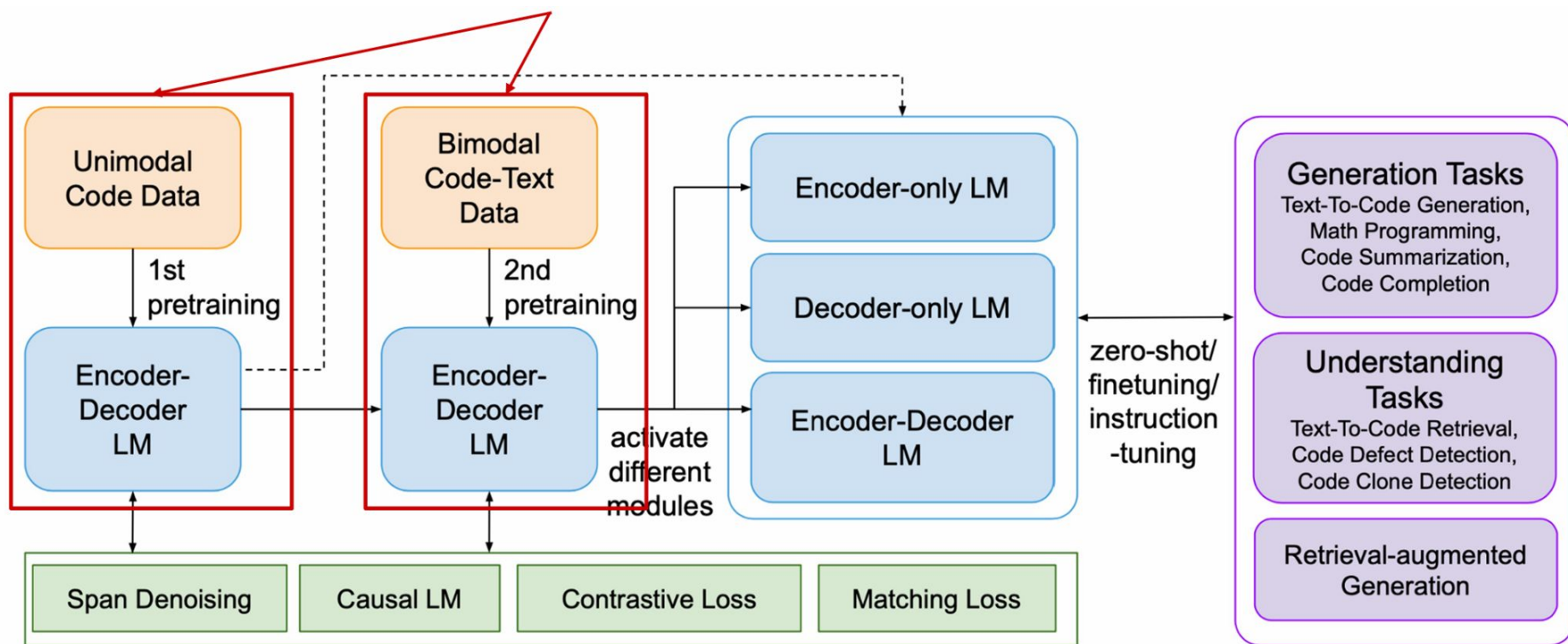
CodeT5+: Can operate in different modes



CodeT5+: Uses several pre-training tasks



CodeT5+: Has two pre-training stages



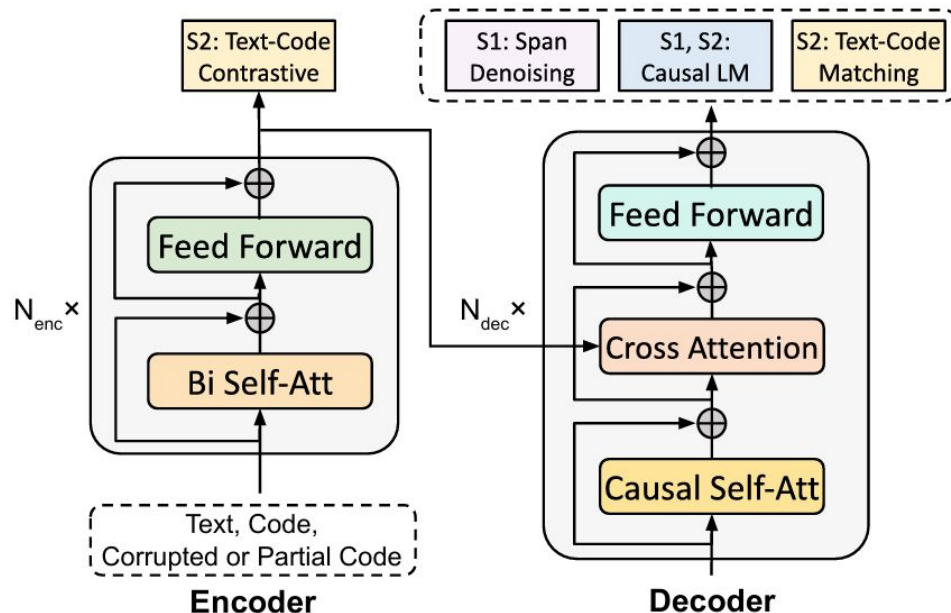
CodeT5+: Stage 1 - Code-only pre-training

- **Goal:** Train model to recover code contexts at different scales
- **Data:** Code from GitHub
- **Tasks:**
 - Span Denoising (15% masked tokens)
 - Causal LM: Partial programs + Complete programs
 - Partial programs: Random select a position to split program to form input and output.
 - Complete programs: input begin token [CLM] output full program.

CodeT5+: Stage 2 - Code and text pre-training

- **Goal:** Train model for cross-modal understanding and generation
- **Data:** CodeSearchNet (Docstring & Code)
- **Tasks:**
 - Contrastive Learning (align feature space of code and text representation)
 - Text-Code Matching (predict if semantics match)
 - Text-Code Causal LM (text-to-code and code-to-text generation)

CodeT5+: Stage 2 - Code and text pre-training



- **Encoder:** encodes contextual representations from either complete, partial or span-masked code/text sequences
- **Decoder:** generates different outputs based on pretraining task

*S1: Stage1 and S2: Stage2

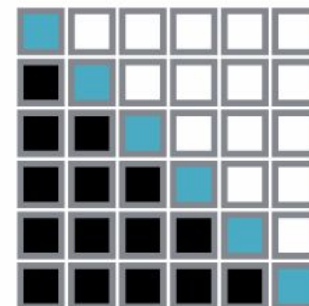
CodeT5+: Results

Code generation: slightly outperforms the CodeGen models it is initialized with

Model	Model size	pass@1	pass@10	pass@100
Closed-source models				
Codex	2.5B	21.4	35.4	59.5
Codex	12B	28.8	46.8	72.3
code-cushman-001	-	33.5	54.3	77.4
code-davinci-002	-	47.0	74.9	92.1
GPT-3.5	-	48.1	-	-
Open-source models				
CodeGen-mono	2B	23.7	36.6	57.0
CodeGen-mono	6B	26.1	42.3	65.8
CodeGen-mono	16B	29.3	49.9	75.0
CodeT5+	220M	12.0	20.7	31.6
CodeT5+	770M	15.5	27.2	42.7
CodeT5+	2B	24.2	38.2	57.8
CodeT5+	6B	28.0	47.2	69.8
CodeT5+	16B	30.9	51.6	76.7

Autoregressive Generation

$$P(X) = \prod_{i=1}^{|X|} P(x_i | x_1, \dots, x_{i-1})$$



OpenAI GPT/GPT2 (Radford et al. 2019)

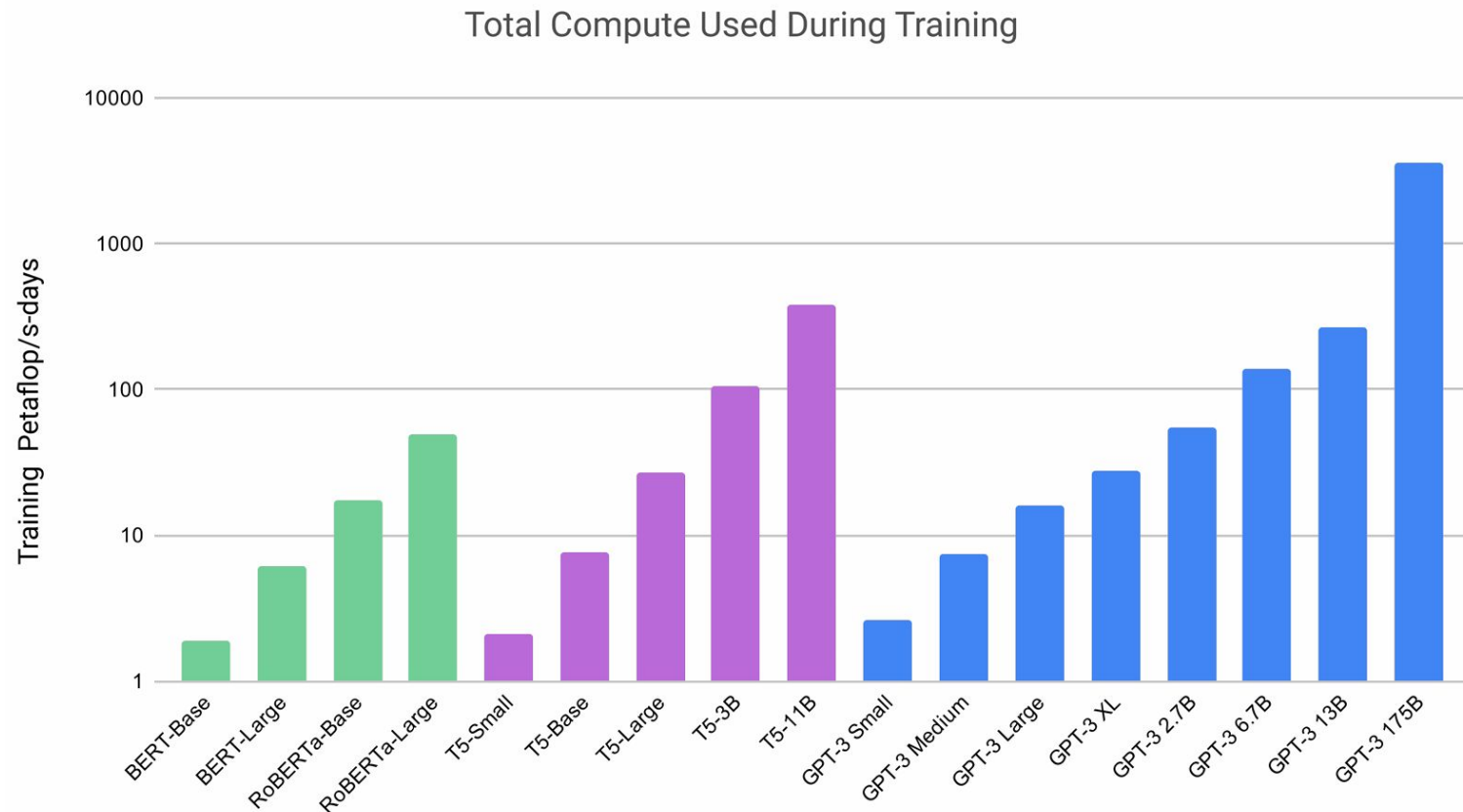
- Very large language models using the Transformer architecture
- Straightforward unidirectional decoder language model, trained on raw text
- GPT2: trained on 40GB of text

	Parameters	Layers	d_{model}
	117M	12	768
approximate size of BERT	345M	24	1024
	762M	36	1280
GPT-2	1542M	48	1600

- By far the largest of these models trained when it came out in March 2019

Pushing the Limits: GPT-3

- 175B parameter model: 96 layers, 96 heads, 12k-dim vectors
- Trained on Microsoft Azure, estimated to cost roughly \$10M



Brown et al. (2020)

- **Model versions:** StarCoderBase and StarCoder (Python)15B
- **Data:** ~816 GB extracted from The Stack v1.2 (Kocetkov et al., 2022)
- **More sources:**
 - Jupyter notebooks
 - Github issues
 - Github commits

Brown et al. (2020)

- **Visual inspection:** Classify the data and randomly sample for human verification.
- **Filtering:** XML, lack Alphabet, YAML, JSON, HTML
- **Deduplication:** MinHashes technique (Broder, 2000)
- **Personally Identifiable Information (PII) redaction:**

- Curate a dataset for PII
- Train a NER model to detect PII
- Replaced the detected PII entities with the special tokens
<NAME>, <EMAIL>, <KEY>, <PASSWORD>

PII type	Count	Recall	Precision
IP_ADDRESS	2526	85%	97%
KEY	308	91%	78%
PASSWORD	598	91%	86%
ID	1702	53%	51%
EMAIL	5470	99%	97%
EMAIL_EXAMPLE	1407		
EMAIL_LICENSE	3141		
NAME	2477	89%	94%
NAME_EXAMPLE	318		
NAME_LICENSE	3105		
USERNAME	780	74%	86%
USERNAME_EXAMPLE	328		
USERNAME_LICENSE	503		
AMBIGUOUS	287		

CodeGemma (CodeGemma Team et al., 2024)

- Trained upon Gemma models (Gemma Team et al., 2024).
- Data: further trained on **500 billion tokens** of primarily English language data from **web documents, mathematics, and code**.
- Objective: Fill-in-the-middle (FIM)

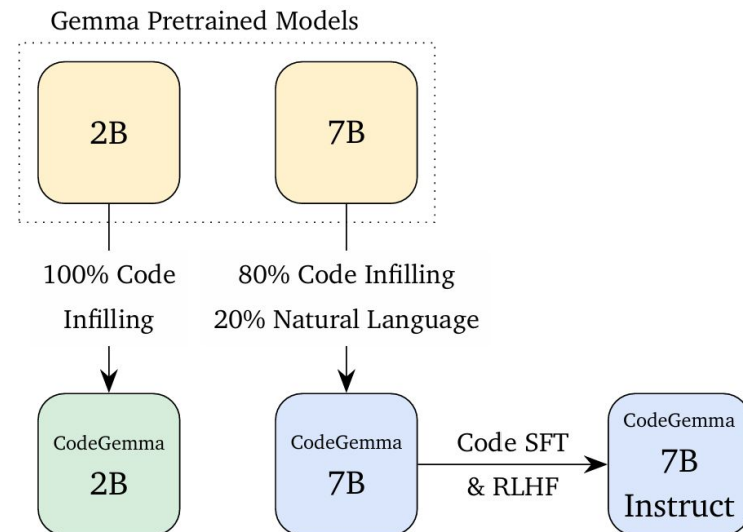


Fig. CodeGemma overview

```
path/to/the/first/file.py
<|fim_prefix|>from typing import List
_
def mean_absolute_deviation(numbers: List[float]) -> float:
    """For a given list of input numbers, calculate Mean Absolute Deviation
    around the mean of this dataset.
    Mean Absolute Deviation is the average absolute difference between each
    element and a centerpoint (mean in this case):
    MAD = average | x - x_mean |
    >>> mean_absolute_deviation([1.0, 2.0, 3.0, 4.0])
    1.0
    """
<|fim_suffix|><|fim_middle|>    return sum(abs(x - mean) for x in numbers) / len(numbers)
<|file_separator|>path/to/the/second/file.py
<|fim_prefix|>...
```

Fig. FIM format for PSM (prefix-suffix-middle)

Fill-in-the-middle (FIM)

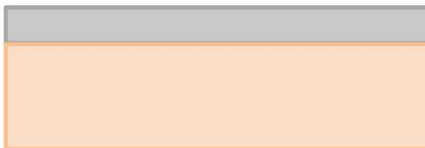
```
def minimize_in_graph(build_loss_fn, num_steps=200, optimizer=None):  
    """ Minimize a loss function using gradient.  
    Args:  
        build_loss_fn: a function that returns a loss tensor for a mini-batch of examples.  
        num_steps: number of gradient descent steps to perform.  
        optimizer: an optimizer to use when minimizing the loss function. If None, will use Adam  
    """  
    optimizer = tf.compat.v1.train.AdamOptimizer(0.1) if optimizer is None else optimizer  
    minimize_op = tf.compat.v1.train.minimize(  
        cond=lambda step: step < num_steps,  
        body=train_loop_body,  
        loop_vars=[tf.constant(0)], return_same_structure=True)[0]  
    return minimize_op
```

Prefix

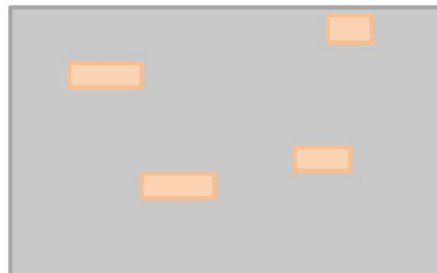
Target

Suffix

“Causal” (L-to-R)



Masked Infilling



“Causal Masking” /
Fill-in-the-Middle (FIM)



Fill-in-the-middle (FIM)

```
def minimize_in_graph(build_loss_fn, num_steps=200, optimizer=None):
```

Prefix

```
    """ Minimize a loss function using gradient.
```

```
    Args:
```

Target

```
        build_loss_fn: a function that returns a loss tensor for a mini-batch of examples.
```

```
        num_steps: number of gradient descent steps to perform.
```

```
        optimizer: an optimizer to use when minimizing the loss function. If None, will use Adam
```

```
    """
```

Suffix

```
    optimizer = tf.compat.v1.train.AdamOptimizer(0.1) if optimizer is None else optimizer
```

```
    minimize_op = tf.compat.v1.train_while_loop(
```

```
        cond=lambda step: step < num_steps,
```

```
        body=train_loop_body,
```

```
        loop_vars=[tf.constant(0)], return_same_structure=True)[0]
```

```
    return minimize_op
```

Bavarian et al. (2022)

- Additional sentinel tokens by <PRE>, <MID>, and <SUF>
- Training format:
 - **PSM**: <PRE> prefix <SUF> suffix <MID> target
 - **SMP**: <SUF> suffix <PRE> prefix <MID> target
- Inference format: <PRE> prefix <SUF> suffix <MID> (PSM)

Fill-in-the-middle (FIM)

```
def minimize_in_graph(build_loss_fn, num_steps=200, optimizer=None):
```

Prefix

```
    """ Minimize a loss function using gradient.
```

```
    Args:
```

Target

```
        build_loss_fn: a function that returns a loss tensor for a mini-batch of examples.
```

```
        num_steps: number of gradient descent steps to perform.
```

```
        optimizer: an optimizer to use when minimizing the loss function. If None, will use Adam
```

```
    """
```

Suffix

```
    optimizer = tf.compat.v1.train.AdamOptimizer(0.1) if optimizer is None else optimizer
```

```
    minimize_op = tf.compat.v1.train_while_loop(
```

```
        cond=lambda step: step < num_steps,
```

```
        body=train_loop_body,
```

```
        loop_vars=[tf.constant(0)], return_same_structure=True)[0]
```

```
    return minimize_op
```

Bavarian et al. (2022)

- Additional sentinel tokens by <PRE>, <MID>, and <SUF>
- Training format:
 - **PSM**: <PRE> prefix <SUF> suffix <MID> target
 - **SMP**: <SUF> suffix <PRE> prefix <MID> target
- Inference format: <PRE> prefix <SUF> suffix <MID> (PSM)

Incoder (Fried, et al. 2022)

Original Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        word_counts = {}
        for line in f:
            for word in line.split():
                if word in word_counts:
                    word_counts[word] += 1
                else:
                    word_counts[word] = 1
    return word_counts
```

Masked Document

```
def count_words(filename: str) -> Dict[str, int]:
    """Count the number of occurrences of each word in the file."""
    with open(filename, 'r') as f:
        <MASK:0> in word_counts:
            word_counts[word] += 1
        else:
            word_counts[word] = 1
    return word_counts
<MASK:0> word_counts = {}
for line in f:
    for word in line.split():
        if word <EOM>
```

- Training Data
 - 600K permissively-licensed repositories from GitHub & GitLab. ~150GB total
 - StackOverflow: questions, answers, comments. ~50GB
- Models
 - Unidirectional, decoder-only Transformer
 - 1B model: ~1 week on 128 V100s
 - 6B model: ~3 weeks on 240 V100s

4. Evaluation in Code Generation

Match-based metrics

Problem

Write a function to find the longest chain which can be formed from the given set of pairs.

Generate code

```
def max_chain_length(pairs):  
    # Sort pairs based on the second  
    element  
    pairs.sort(key=lambda x: x[1])  
  
    # Initialize variables  
    current_end = float('-inf')  
    max_chain = 0  
  
    # Iterate through pairs  
    for pair in pairs:  
        if pair[0] > current_end:  
            current_end = pair[1]  
            max_chain += 1  
  
    return max_chain
```

Solution

```
class Pair(object):  
    def __init__(self, a, b):  
        self.a = a  
        self.b = b  
    def max_chain_length(arr, n):  
        max = 0  
        mcl = [1 for i in range(n)]  
        for i in range(1, n):  
            for j in range(0, i):  
                if (arr[i].a > arr[j].b and  
                    mcl[i] < mcl[j] + 1):  
                    mcl[i] = mcl[j] + 1  
        for i in range(n):  
            if (max < mcl[i]):  
                max = mcl[i]  
        return max
```

- Objective: The generated code can fulfill the requirements specified in the input problem (NL)
 - Compare generated code with the solution

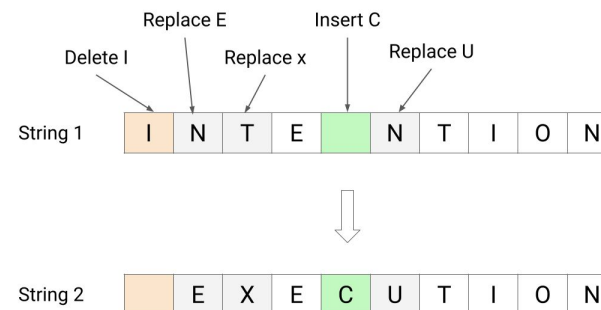
Match-based metrics

- ❖ **BLEU score:** is an n-gram precision-based metric that will be higher when more subparts of the predicted code match the provided reference.

$$\text{BLEU} = \underbrace{\min\left(1, \exp\left(1 - \frac{\text{reference-length}}{\text{output-length}}\right)\right)}_{\text{brevity penalty}} \underbrace{\left(\prod_{i=1}^4 \text{precision}_i\right)^{1/4}}_{\text{n-gram overlap}}$$

- ❖ **Edit similarity** (Edit/Levenshtein distance): measures the minimum number of single-character edits (insertions, deletions, or substitutions) required to transform one string into the other.

Edit Distance Example



Dynamic programming

Match-based metric: CodeBLEU (Ren et al. 2020)

- ❖ **CodeBLEU**: considers information from not only the shallow (n-gram) match, but also the syntactic match (AST) and the semantic match (Data-flow)

Machine translation:

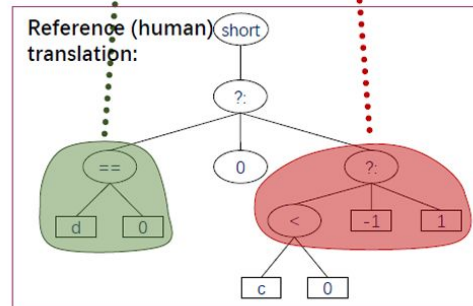
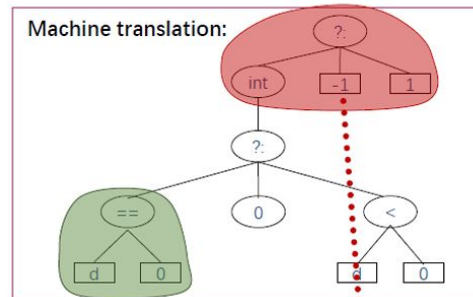
```
public static int Sign ( double d )
{
    return ( (int) ) ( ( d == 0 ) ? 0 : ( d < 0 ) ) ?
    -1 : 1;
}
```

1.0 1.0 0.7 0.5

Reference (human) translation:

```
public static short Sign ( double d )
{
    return ( short ) ( ( d == 0 ) ? 0 : ( d < 0 ) ?
    -1 : 1 );
}
```

Weighted N-Gram Match



Syntactic AST Match

[('d', 7, 'comesFrom', [], []),
('d', 16, 'comesFrom', ['d'], [7]),
('d', 24, 'comesFrom', ['d'], [7])]

Machine translation:

```
public static int Sign ( double d )
{
    return ( (int) ) ( ( d == 0 ) ? 0 : ( d < 0 ) ) ?
    -1 : 1;
}
```

Reference (human) translation:

```
public static short Sign ( double c )
{
    return ( short ) ( ( c == 0 ) ? 0 : ( c < 0 ) ? -
    1 : 1 );
}
```

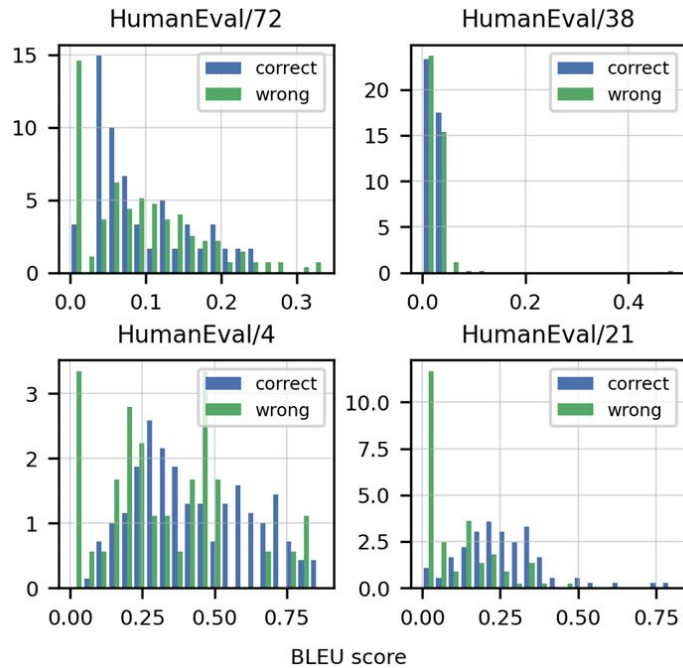
Semantic Data-flow Match

$$\text{CodeBLEU} = \alpha \cdot \text{N-Gram Match (BLEU)} + \beta \cdot \text{Weighted N-Gram Match} + \gamma \cdot \text{Syntactic AST Match} + \delta \cdot \text{Semantic Data-flow Match}$$

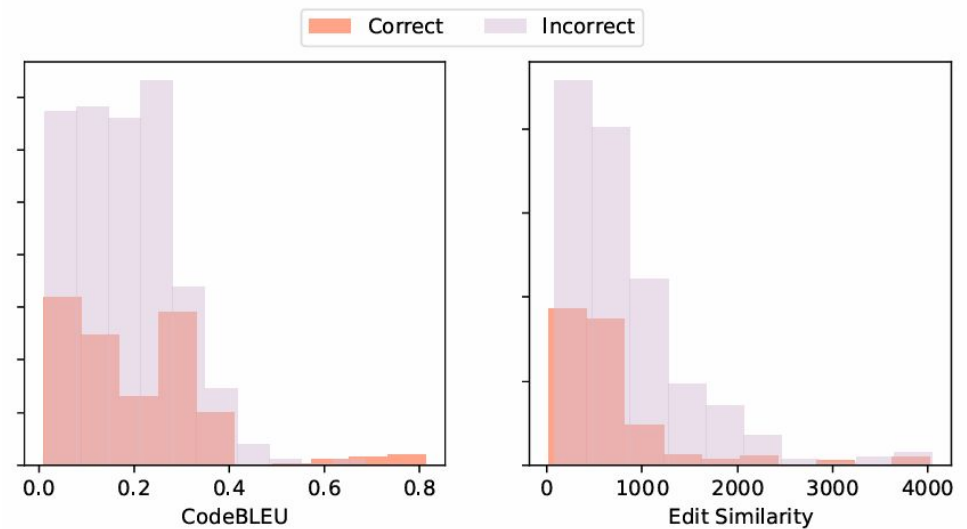
2 cái cuối liên quan đến cấu trúc của code

Match-based metrics: Weaknesses

Do not reflect functional correctness



Chen et al (2021)



Hai et al (2024)

Execution-based metrics

Problem

Write a function to find the longest chain which can be formed from the given set of pairs.

Generate code

```
def max chain length (pairs):  
    # Sort pairs based on the second  
    element  
    pairs.sort(key=lambda x: x[1])  
  
    # Initialize variables  
    current end = float('-inf')  
    max_chain = 0  
  
    # Iterate through pairs  
    for pair in pairs:  
        if pair[0] > current end:  
            current end = pair[1]  
            max_chain += 1  
  
    return max_chain
```

Test cases

```
assert max chain length([Pair(5,  
24), Pair(15, 25),Pair(27, 40),  
Pair(50, 60)], 4) == 3  
assert max chain length([Pair(1, 2),  
Pair(3, 4),Pair(5, 6), Pair(7, 8)],  
4) == 4  
assert max chain length([Pair(19,  
10), Pair(11, 12),Pair(13, 14),  
Pair(15, 16), Pair(31, 54)], 5) == 5
```

- Objective: The generated code can fulfill the requirements specified in the input problem (NL)
 - Generated solutions must pass all test cases

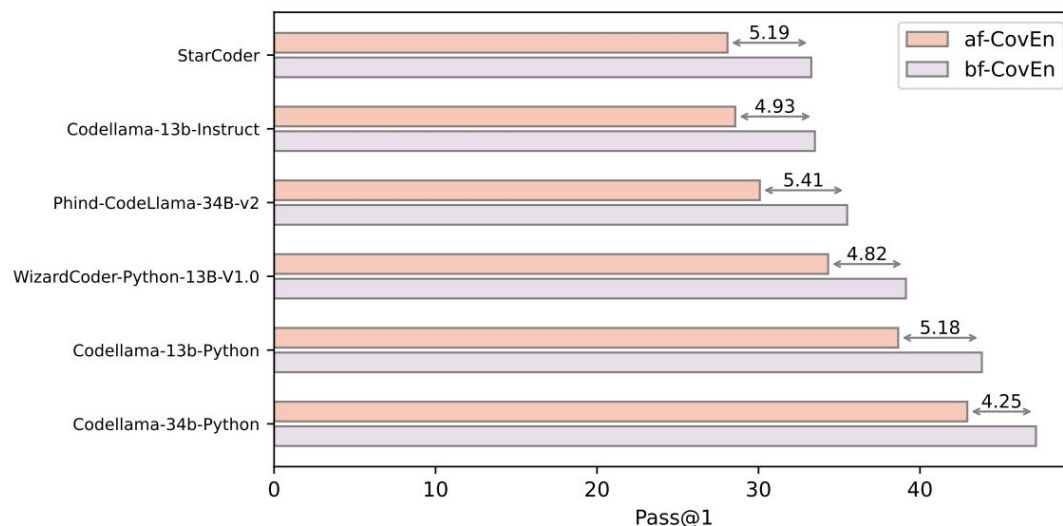
Execution-based metric: Pass@k (Chen et al, 2021)

- The $pass@k$ metric is defined as the probability that at least one of the top k-generated code samples for a problem passes the unit tests.

$$pass@k := \mathbb{E}_{\text{problems}} \left[1 - \frac{C(n - c, k)}{C(n, k)} \right]$$

1 số nhược điểm của execution-base:
không có testcase
không đánh giá được syntax

- Require significant effort to create strong test cases
 - Practical coding scenarios usually do not have test cases
- Poor solutions can still pass the test (workaround solution, solution contains redundant statements, v.v)

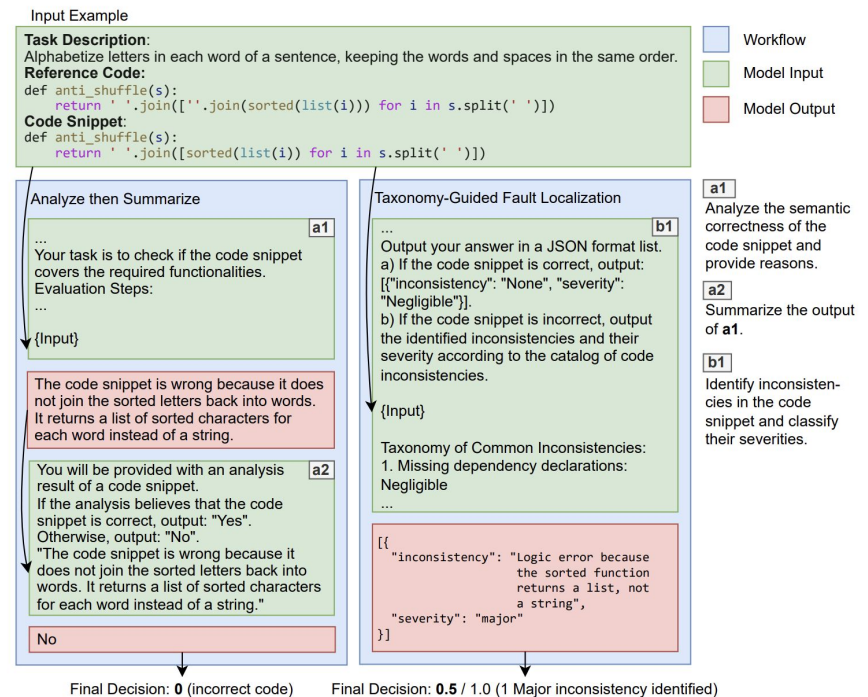
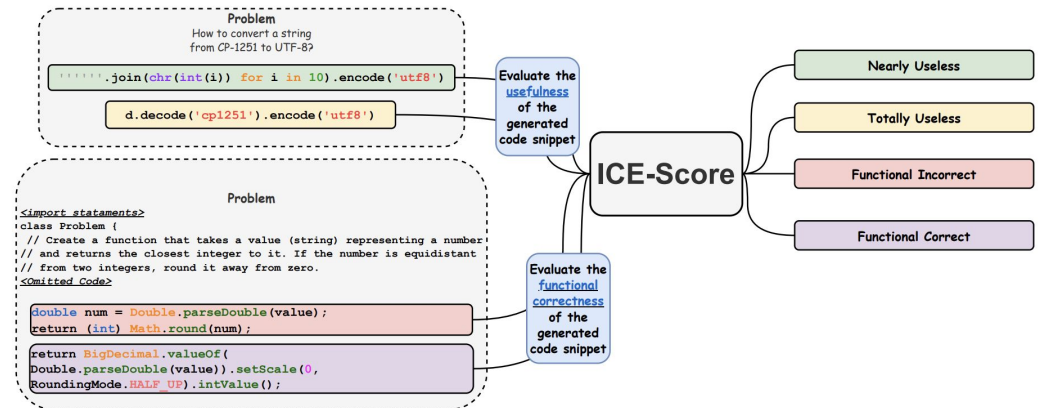


Can we combine both execution-based and match based metrics?

Fig. Stronger test cases impact results significantly (Hai et al, 2024)

LLM as a judge

- Use a strong LLM to evaluate generated solutions
- **ICE-Score**
(Zhuo, 2023)
- **CodeJudge**
(Tong, W., & Zhang, T. 2024)



A large, stylized graphic on the left side of the slide. It consists of a red background with a circular pattern of white dots of varying sizes, creating a sense of depth and movement. The word "HUST" is written in white, bold, sans-serif capital letters in the center of this graphic.

HUST

THANK YOU !